Large Language Model

Transformer, Attention, Pretraining, Fine-tuning, Prompting, RAG

Hanwha Life Vietnam

Phiên bản v1.0.0

Ngày 12 tháng 10 năm 2026

Mục lục

1 Tổng quan **2**
- 1.1 Triết lý trình bày . . . 2
- 1.2 Cách đọc tài liệu . . . 2
- 1.3 Nội dung các chương . . . 2
- 1.4 Nguồn và quy ước . . . 3

2 Mô hình Transformer **4**
- 2.1 NLP và sự trỗi dậy của LLM . . . 4
  - 2.1.1 Mô hình ngôn ngữ lớn là gì . . . 5
  - 2.1.2 Vì sao xử lý ngôn ngữ lại khó . . . 6
- 2.2 Transformer làm được gì: hàm `pipeline` . . . 6
  - 2.2.1 Các pipeline theo loại dữ liệu . . . 6
  - 2.2.2 Phân loại không mẫu (zero-shot) . . . 7
  - 2.2.3 Sinh văn bản . . . 7
  - 2.2.4 Điền chỗ trống, nhận dạng thực thể, hỏi đáp . . . 7
- 2.3 Transformer hoạt động thế nào . . . 8
  - 2.3.1 Một chút lịch sử . . . 8
  - 2.3.2 Transformer là mô hình ngôn ngữ, và chúng rất lớn . . . 9
  - 2.3.3 Học chuyển giao: pretraining và fine-tuning . . . 9
  - 2.3.4 Kiến trúc tổng quát: Encoder và Decoder . . . 10
  - 2.3.5 Lớp attention: trái tim của Transformer . . . 10
  - 2.3.6 Kiến trúc gốc và che attention . . . 11
- 2.4 Cách Transformer giải các bài toán . . . 12
  - 2.4.1 Sinh văn bản với mô hình kiểu GPT . . . 12
  - 2.4.2 Phân loại, gán nhãn token và hỏi đáp với BERT . . . 12
  - 2.4.3 Tóm tắt và dịch với encoder-decoder . . . 13
  - 2.4.4 Vượt ra ngoài văn bản . . . 13

3 Dùng thư viện Transformers **14**
- 3.1 Bên trong một pipeline . . . 14
  - 3.1.1 Bước 1 – Tiền xử lý bằng tokenizer . . . 14
  - 3.1.2 Bước 2 – Đưa qua mô hình . . . 15
  - 3.1.3 Bước 3 – Hậu xử lý . . . 15
- 3.2 Mô hình: tạo, lưu, chia sẻ . . . 16
- 3.3 Tokenizer: biến văn bản thành số . . . 16
  - 3.3.1 Mã hoá và giải mã . . . 17
  - 3.3.2 Token đặc biệt . . . 17
- 3.4 Xử lý nhiều câu một lúc . . . 17
  - 3.4.1 Mô hình mong đợi một batch . . . 17
  - 3.4.2 Đệm (padding) và attention mask . . . 18
  - 3.4.3 Chuỗi quá dài . . . 18

3.5 Gộp tất cả: tokenizer làm hết . . . 18

4 Tinh chỉnh mô hình đã huấn luyện 20
4.1 Huấn luyện trên một batch . . . 20
4.2 Chuẩn bị dữ liệu . . . 20
4.2.1 Nạp dữ liệu từ Hub . . . 21
4.2.2 Token hoá cặp câu . . . 21
4.2.3 Áp token hoá cho cả tập với `map` . . . 21
4.2.4 Đệm động . . . 21
4.3 Huấn luyện với `Trainer` . . . 23
4.3.1 Khai báo tham số và mô hình . . . 23
4.3.2 Tạo `Trainer` và huấn luyện . . . 23
4.3.3 Đánh giá . . . 23
4.3.4 Vài tính năng huấn luyện nâng cao . . . 24
4.4 Vòng lặp huấn luyện thủ công . . . 24

5 Chia sẻ mô hình và tokenizer 25
5.1 Dùng mô hình tiền huấn luyện . . . 25
5.2 Chia sẻ mô hình lên Hub . . . 25
5.2.1 Cách 1 – API `push_to_hub` . . . 26
5.2.2 Cách 2 – Thư viện `huggingface_hub` . . . 26
5.2.3 Cách 3 – Giao diện web và git . . . 27

6 Thư viện Datasets 28
6.1 Cắt gọt và làm sạch dữ liệu . . . 29
6.1.1 Đổi tên cột và chuẩn hoá nhãn . . . 30
6.1.2 Tạo cột mới . . . 30
6.2 Sức mạnh của `map(batched=True)` . . . 30
6.2.1 Một ví dụ sinh ra nhiều đặc trưng . . . 31
6.3 Qua lại với Pandas . . . 31
6.4 Tạo tập validation và lưu dữ liệu . . . 31
6.5 Dữ liệu lớn: ánh xạ bộ nhớ và truyền dòng . . . 32

7 Thư viện Tokenizers 33
7.1 Huấn luyện tokenizer mới từ một tokenizer cũ . . . 33
7.1.1 Chuẩn bị kho dữ liệu . . . 33
7.1.2 Huấn luyện và lưu . . . 34
7.2 Siêu năng lực của tokenizer nhanh . . . 34
7.2.1 Dựng lại pipeline phân loại token . . . 35
7.2.2 Gộp thực thể . . . 35

8 Các bài toán NLP kinh điển 37
8.1 Công thức chung . . . 38
8.2 Phân loại token . . . 38
8.3 Mô hình ngôn ngữ che từ . . . 38
8.4 Tóm tắt và dịch . . . 39
8.5 Tiền huấn luyện mô hình ngôn ngữ nhân quả . . . 39
8.6 Hỏi đáp . . . 39

9 Khi cần trợ giúp **41**
9.1 Đọc thông báo lỗi Python . . . 41
9.2 Gỡ lỗi pipeline suy luận . . . 43
9.3 Gỡ lỗi quá trình huấn luyện . . . 43
9.3.1 Kiểm tra dữ liệu . . . 43
9.3.2 Từ dataset tới dataloader . . . 43
9.3.3 Đưa batch qua mô hình . . . 44
9.3.4 Một bước tối ưu và đánh giá . . . 44
9.4 Gỡ lỗi "im lặng": huấn luyện xong nhưng mô hình kém . . . 44
9.5 Hỏi cộng đồng cho hiệu quả . . . 44

10 Dựng và chia sẻ demo **46**
10.1 Demo đầu tiên . . . 46
10.1.1 Gắn mô hình thật vào demo . . . 47
10.2 Hiểu sâu lớp `Interface` . . . 47
10.2.1 Ví dụ với âm thanh . . . 47
10.2.2 Nhiều đầu vào và đầu ra . . . 48
10.2.3 Phương thức `launch` . . . 48
10.3 Hoàn thiện và chia sẻ demo . . . 49
10.3.1 Làm đẹp giao diện . . . 49
10.3.2 Hai cách chia sẻ . . . 49

11 Tuyển chọn dữ liệu chất lượng **51**
11.1 Dựng và kết nối Argilla . . . 51
11.2 Nạp và cấu hình dữ liệu . . . 51
11.2.1 Tạo dataset và nạp bản ghi . . . 52
11.3 Chú thích dữ liệu . . . 52
11.3.1 Thống nhất bằng guidelines . . . 52
11.3.2 Phân phối công việc . . . 52
11.3.3 Chú thích từng bản ghi . . . 53

12 Tinh chỉnh mô hình ngôn ngữ lớn **54**
12.1 Mẫu hội thoại (chat templates) . . . 54
12.2 Tinh chỉnh có giám sát (SFT) . . . 55
12.2.1 Khi nào nên dùng SFT . . . 55
12.2.2 Huấn luyện với `SFTTrainer` . . . 55
12.2.3 Đọc đường cong loss . . . 55
12.3 LoRA: tinh chỉnh tiết kiệm tham số . . . 56
12.3.1 Cấu hình LoRA với PEFT . . . 56
12.3.2 Dùng LoRA trong `SFTTrainer` và gộp adapter . . . 56

13 Mô hình suy luận **58**
13.1 Học tăng cường là gì . . . 58
13.2 Vì sao RL quan trọng với LLM . . . 58
13.3 GRPO và vì sao chọn nó . . . 59
13.4 Hiện thực GRPO với TRL . . . 59
13.4.1 Kích thước nhóm . . . 60
13.4.2 Thiết kế hàm thưởng . . . 60

Chương 1

Tổng quan

Tài liệu này là bộ ghi chú học tập về **mô hình ngôn ngữ lớn** (Large Language Model – LLM) và họ mô hình **Transformer**, biên soạn dựa trên khoá *LLM Course* của Hugging Face. Tài liệu đi theo hướng **high-level, trực quan và thực chiến**: ưu tiên hiểu ý tưởng và biết cách dùng thư viện, thay vì tự cài đặt thuật toán từ đầu.

1.1 Triết lý trình bày

- **Trực giác trước, chi tiết sau** – mỗi khái niệm được giải thích bằng ví dụ đời thường và sơ đồ. Thay vì công thức toán, tài liệu mô tả bằng lời: nói rõ đại lượng nào vào, đại lượng nào ra, và vì sao.
- **Thực chiến bằng thư viện** – code ví dụ dùng hệ sinh thái Hugging Face (`transformers`, `datasets`, `tokenizers`) cùng PyTorch. Mục tiêu là chạy được việc thật, không phải viết lại Transformer từ con số không.
- **Sơ đồ hoá** – kiến trúc, luồng xử lý và quan hệ giữa các thành phần đều được minh hoạ bằng sơ đồ để dễ nhớ và dễ tra cứu.

1.2 Cách đọc tài liệu

Mỗi chương gồm ba lớp nội dung đan xen:

- **Lý thuyết** – giải thích bằng ví dụ và sơ đồ, hạn chế ký hiệu toán.
- **Code** – ví dụ Python đặt trong khung, có chú thích tiếng Việt, kèm giải thích từng bước dùng để làm gì.
- **Áp dụng** – checklist, bảng so sánh, lưu ý khi đưa vào dự án thực tế.

1.3 Nội dung các chương

Chương	Nội dung chính
Ch. 2: Mô hình Transformer	NLP và LLM là gì, Transformer làm được gì, cơ chế attention, kiến trúc encoder/decoder, cách mô hình giải các bài toán.
Ch. 3: Dùng thư viện Transformers	bên trong hàm `pipeline`, mô hình, tokenizer, xử lý batch nhiều câu.

Bảng 1.1 – *tiếp theo*

Chương	**Nội dung chính**
Ch. 4: Tinh chỉnh mô hình đã huấn luyện	chuẩn bị dữ liệu, lớp `Trainer`, vòng lặp huấn luyện thủ công, tăng tốc với Accelerate.
Ch. 5: Chia sẻ mô hình và tokenizer	Model Hub, tải lên mô hình, viết thẻ mô hình (model card).
Ch. 6: Thư viện Datasets	nạp, xử lý, lọc dữ liệu lớn, làm việc với dữ liệu không nằm trên Hub.
Ch. 7: Thư viện Tokenizers	huấn luyện tokenizer mới, các thuật toán tách từ phụ (subword), tokenizer nhanh.
Ch. 8: Các bài toán NLP kinh điển	phân loại token, mô hình ngôn ngữ, dịch, tóm tắt, hỏi đáp.
Ch. 9: Khi cần trợ giúp	đọc thông báo lỗi, gỡ lỗi pipeline, hỏi đúng cách trên diễn đàn.
Ch. 10: Dựng và chia sẻ demo	xây giao diện demo với Gradio, chia sẻ công khai.
Ch. 11: Tuyển chọn dữ liệu chất lượng	tạo và đánh giá bộ dữ liệu tốt cho huấn luyện.
Ch. 12: Tinh chỉnh LLM	định dạng hội thoại, giám sát (SFT), tinh chỉnh tiết kiệm tham số (LoRA).
Ch. 13: Mô hình suy luận	xây mô hình biết suy luận, học tăng cường cho suy luận (GRPO).

1.4 Nguồn và quy ước

Nội dung được biên soạn và diễn giải lại từ khoá học *LLM Course* của Hugging Face (`https://huggingface.co/learn/llm-course`). Các đoạn mô tả đã được viết lại bằng tiếng Việt cho phù hợp mục đích học tập; phần code giữ nguyên cú pháp cần thiết để chạy được. Mọi con số minh hoạ (ví dụ điểm số, nhãn dự đoán) đều lấy trực tiếp từ ví dụ trong tài liệu gốc.

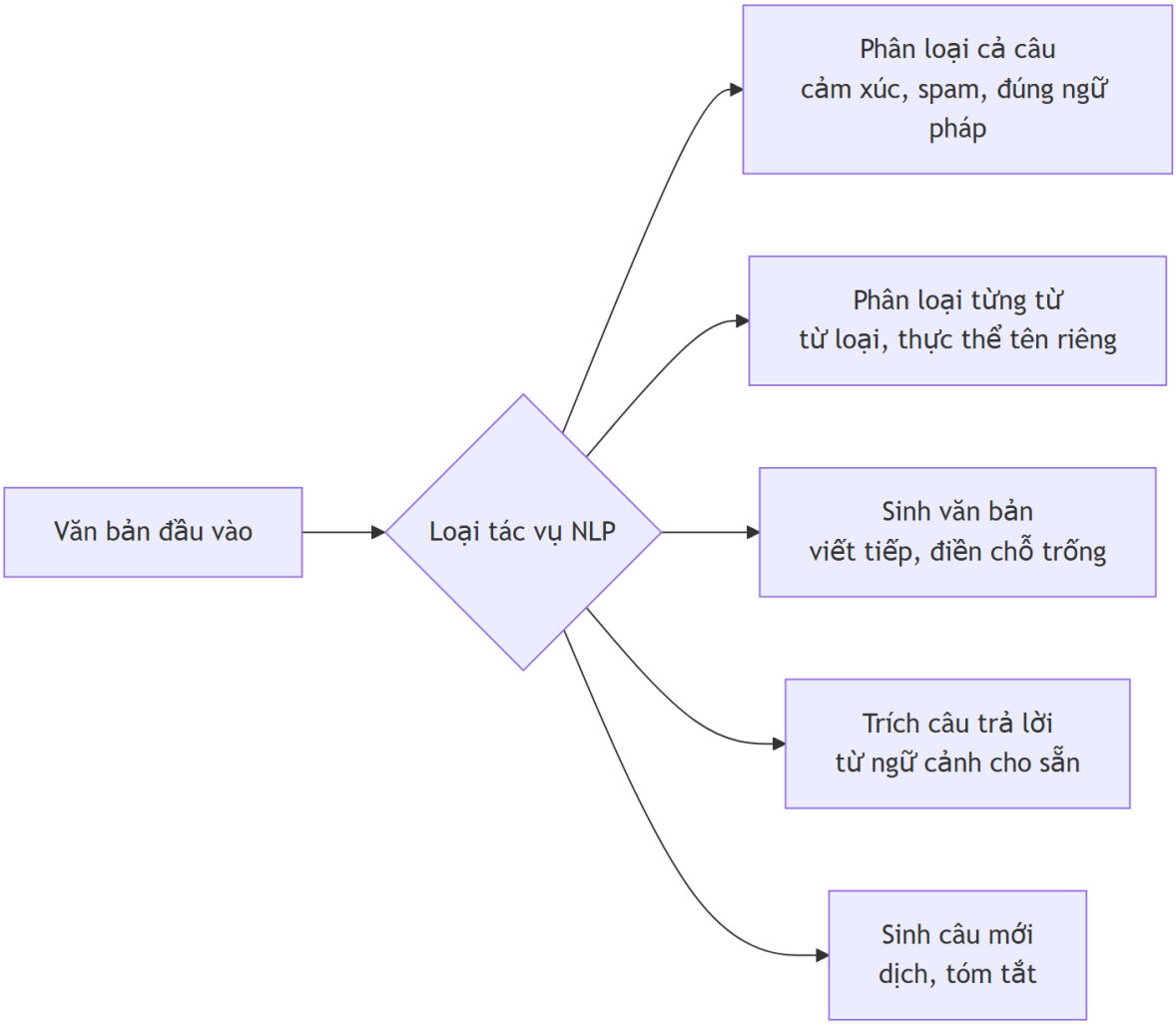
Chương 2

Mô hình Transformer

Chương này trả lời ba câu hỏi nền tảng: xử lý ngôn ngữ tự nhiên (NLP) và mô hình ngôn ngữ lớn (LLM) là gì, mô hình Transformer làm được những việc gì, và bên trong nó hoạt động ra sao. Mục tiêu là xây một bức tranh trực giác đủ vững để các chương sau đi vào thực hành.

2.1 NLP và sự trỗi dậy của LLM

Xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) là lĩnh vực giao giữa ngôn ngữ học và học máy, tập trung vào việc giúp máy hiểu ngôn ngữ con người. Điểm mấu chốt: không chỉ hiểu từng từ riêng lẻ mà phải hiểu *ngữ cảnh* của các từ đó. Các tác vụ NLP thường gặp được tóm trong Hình 2.1.



Hình 2.1: Các nhóm tác vụ NLP phổ biến.

NLP không giới hạn ở văn bản: nó còn xử lý tiếng nói (tạo bản ghi âm thanh) và thị giác máy tính (mô tả nội dung ảnh).

2.1.1 Mô hình ngôn ngữ lớn là gì

Những năm gần đây, NLP được định hình lại bởi **mô hình ngôn ngữ lớn** (LLM) – các kiến trúc như GPT (Generative Pre-trained Transformer) hay Llama. Một LLM là mô hình AI được huấn luyện trên lượng văn bản khổng lồ, có khả năng hiểu và sinh văn bản giống người, nhận ra quy luật trong ngôn ngữ, và làm được nhiều tác vụ ngôn ngữ mà không cần huấn luyện riêng cho từng tác vụ.

Đặc trưng của LLM:

- **Quy mô** – hàng triệu đến hàng trăm tỉ tham số.
- **Năng lực tổng quát** – làm nhiều việc không cần huấn luyện chuyên biệt.
- **Học trong ngữ cảnh** (in-context learning) – học từ chính các ví dụ đặt trong câu lệnh (prompt).
- **Khả năng nổi lên** (emergent abilities) – khi mô hình lớn dần, xuất hiện những năng lực không được lập trình trước.

LLM đã chuyển mô thức từ *xây mô hình chuyên biệt cho từng tác vụ* sang *dùng một mô hình lớn chung*, rồi điều khiển bằng prompt hoặc tinh chỉnh. Dù mạnh, LLM vẫn có giới hạn quan trọng: **ảo giác**

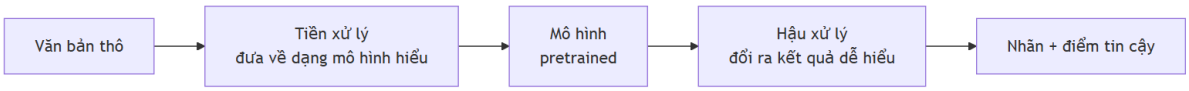
(tự tin đưa thông tin sai), **thiếu hiểu biết thực sự** về thế giới (chỉ vận hành trên quy luật thống kê), **thiên lệch** theo dữ liệu huấn luyện, **cửa sổ ngữ cảnh giới hạn**, và **tốn tài nguyên tính toán**.

2.1.2 Vì sao xử lý ngôn ngữ lại khó

Máy tính không xử lý thông tin như con người. Khi đọc câu "Tôi đói", con người hiểu ngay; cho hai câu "Tôi đói" và "Tôi buồn", ta dễ nhận ra chúng gần nhau đến mức nào. Với mô hình học máy, những việc này khó hơn nhiều: văn bản phải được biến đổi về dạng mà mô hình học được. Vì ngôn ngữ phức tạp, cách biểu diễn văn bản là cả một hướng nghiên cứu lớn. Ngay cả khi đã có LLM, nhiều thách thức căn bản vẫn còn: hiểu sự nhập nhằng, ngữ cảnh văn hoá, châm biếm và hài hước.

2.2 Transformer làm được gì: hàm `pipeline`

Công cụ đầu tiên và cơ bản nhất của thư viện `transformers` là hàm `pipeline`. Nó nối mô hình với các bước tiền xử lý và hậu xử lý cần thiết, nên ta chỉ cần đưa văn bản vào và nhận kết quả có nghĩa ra. Quá trình bên trong gồm ba bước, minh hoạ ở Hình 2.2.



Hình 2.2: Ba bước xử lý bên trong một `pipeline`.

Ví dụ phân tích cảm xúc. Lần đầu tạo đối tượng, mô hình (đã được tinh chỉnh sẵn cho tác vụ này) sẽ được tải về và lưu cache; các lần sau dùng lại bản cache.

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a HuggingFace course my whole life.")
# [{'label': 'POSITIVE', 'score': 0.9598047137260437}]
```

Có thể đưa nhiều câu một lúc:

```
classifier(
    ["I've been waiting for a HuggingFace course my whole life.",
     "I hate this so much!"]
)
# [{'label': 'POSITIVE', 'score': 0.9598047137260437},
#  {'label': 'NEGATIVE', 'score': 0.9994558095932007}]
```

2.2.1 Các pipeline theo loại dữ liệu

Hàm `pipeline` hỗ trợ nhiều loại dữ liệu. Tài liệu này tập trung vào văn bản, nhưng điểm qua để thấy độ phủ của kiến trúc Transformer.

Loại	Một số pipeline tiêu biểu
Văn bản	sinh văn bản, phân loại văn bản, tóm tắt, dịch, phân loại không mẫu (zero-shot), trích đặc trưng.
Ảnh	mô tả ảnh thành chữ, phân loại ảnh, phát hiện vật thể.
Âm thanh	nhận dạng tiếng nói, phân loại âm thanh, chuyển chữ thành tiếng.
Đa phương thức	trả lời câu hỏi về một ảnh dựa trên prompt.

Bảng 2.1: Các nhóm pipeline theo loại dữ liệu.

2.2.2 Phân loại không mẫu (zero-shot)

Trong dự án thực tế, gán nhãn dữ liệu rất tốn công. Pipeline `zero-shot-classification` cho phép tự chỉ định danh sách nhãn, không cần huấn luyện trước trên nhãn đó.

```
from transformers import pipeline

classifier = pipeline("zero-shot-classification")
classifier(
    "This is a course about the Transformers library",
    candidate_labels=["education", "politics", "business"],
)
# labels: ['education', 'business', 'politics']
# scores: [0.8446, 0.1120, 0.0434]
```

Gọi là "zero-shot" vì không cần tinh chỉnh mô hình trên dữ liệu của bạn: nó trả thẳng điểm xác suất cho bất kỳ danh sách nhãn nào.

2.2.3 Sinh văn bản

Ý tưởng: bạn đưa một prompt, mô hình tự viết tiếp. Vì có yếu tố ngẫu nhiên nên kết quả mỗi lần chạy có thể khác nhau. Hai tham số hay dùng là `num_return_sequences` (số chuỗi sinh ra) và `max_length` (độ dài tối đa).

```
from transformers import pipeline

generator = pipeline("text-generation", model="HuggingFaceTB/SmolLM2-360M")
generator(
    "In this course, we will teach you how to",
    max_length=30,
    num_return_sequences=2,
)
```

Có thể chọn bất kỳ mô hình nào từ Model Hub bằng tham số `model`, kể cả mô hình đa ngôn ngữ.

2.2.4 Điền chỗ trống, nhận dạng thực thể, hỏi đáp

Điền chỗ trống (`fill-mask`): mô hình đoán từ còn thiếu ở vị trí `<mask>`. Tham số `top_k` quy định số phương án hiển thị.

```
from transformers import pipeline

unmasker = pipeline("fill-mask")
unmasker("This course will teach you all about <mask> models.", top_k=2)
# ' mathematical' (0.1962), ' computational' (0.0405)
```

Nhận dạng thực thể tên riêng (NER): tìm phần nào của câu là người (PER), tổ chức (ORG), địa điểm (LOC)… Tham số `grouped_entities=True` gộp các mảnh cùng một thực thể lại với nhau.

```
from transformers import pipeline

ner = pipeline("ner", grouped_entities=True)
ner("My name is Sylvain and I work at Hugging Face in Brooklyn.")
# Sylvain -> PER, Hugging Face -> ORG, Brooklyn -> LOC
```

Hỏi đáp (`question-answering`): *trích* câu trả lời từ ngữ cảnh cho sẵn chứ không tự sinh ra.

```
from transformers import pipeline

question_answerer = pipeline("question-answering")
question_answerer(
    question="Where do I work?",
    context="My name is Sylvain and I work at Hugging Face in Brooklyn",
)
# answer: 'Hugging Face'
```

Tóm tắt (`summarization`) và **dịch** (`translation`) cũng nhận `max_length`/`min_length`. Với dịch, cách dễ nhất là chọn mô hình phù hợp trên Hub, ví dụ `Helsinki-NLP/opus-mt-fr-en` để dịch Pháp sang Anh.

> **Lưu ý thực tế**
>
> Các pipeline ở chương này chủ yếu để minh hoạ. Chúng được lập trình cho tác vụ cụ thể và không làm được biến thể khác. Chương 3 sẽ mổ xẻ bên trong `pipeline` và cách tuỳ biến nó.

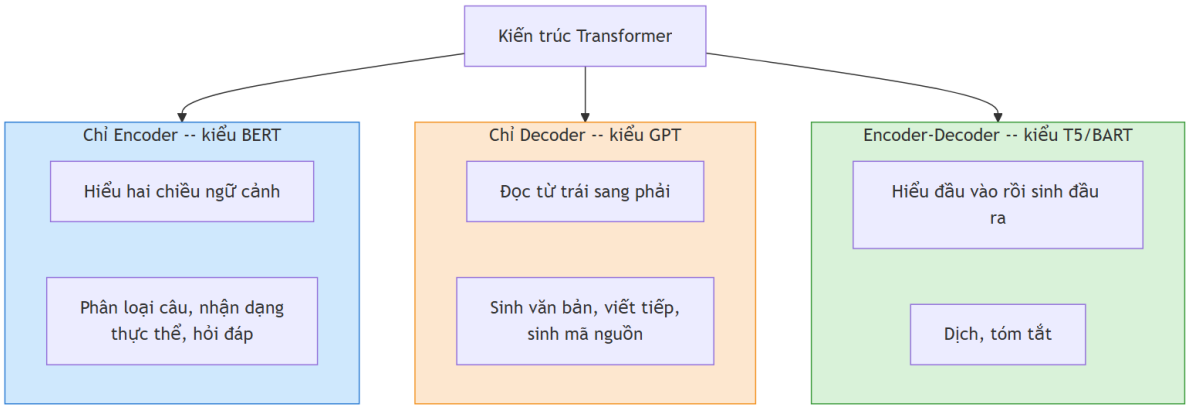
2.3 Transformer hoạt động thế nào

2.3.1 Một chút lịch sử

Kiến trúc Transformer ra đời tháng 6/2017, ban đầu nhắm vào bài toán dịch máy. Sau đó là một loạt mô hình có ảnh hưởng:

- **2018 – GPT**: Transformer tiền huấn luyện đầu tiên, tinh chỉnh được cho nhiều tác vụ NLP.
- **2018 – BERT**: mô hình lớn tạo biểu diễn câu tốt hơn.
- **2019 – GPT-2, T5**: GPT-2 lớn hơn GPT; T5 theo hướng chuỗi-sang-chuỗi đa tác vụ.
- **2020 – GPT-3**: làm tốt nhiều tác vụ mà không cần tinh chỉnh (zero-shot).
- **2022–2024 – InstructGPT, Llama, Mistral, Gemma 2, SmolLM2**: hướng tới bám sát chỉ dẫn, đa ngôn ngữ, suy luận nhanh, và cả mô hình nhỏ gọn cho thiết bị di động.

Dù danh sách còn dài, có thể chia Transformer thành ba họ, xem Hình 2.3.



Hình 2.3: Ba họ mô hình Transformer và thế mạnh của mỗi họ.

2.3.2 Transformer là mô hình ngôn ngữ, và chúng rất lớn

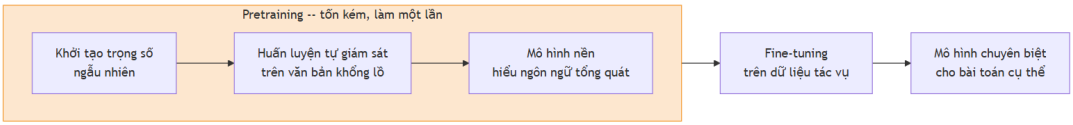
Mọi mô hình kể trên đều được huấn luyện như **mô hình ngôn ngữ**: học trên khối văn bản thô theo kiểu **tự giám sát** (self-supervised) – mục tiêu huấn luyện tự tính ra từ chính dữ liệu đầu vào, không cần người gán nhãn. Hai kiểu huấn luyện tiêu biểu:

- **Mô hình ngôn ngữ nhân quả** (causal language modeling): đoán từ tiếp theo dựa trên các từ đã đọc trước đó. Kết quả chỉ phụ thuộc đầu vào ở quá khứ và hiện tại, không dùng tương lai. Đây là cách GPT học.
- **Mô hình ngôn ngữ che từ** (masked language modeling): che ngẫu nhiên một số từ rồi bắt mô hình đoán lại, nhờ đó học được ngữ cảnh *hai chiều*. Đây là cách BERT học.

Chiến lược chung để mạnh hơn là tăng kích thước mô hình và lượng dữ liệu tiền huấn luyện. Hệ quả: huấn luyện một mô hình lớn rất tốn thời gian, tiền bạc và có cả tác động môi trường (lượng khí thải carbon). Vì thế việc **chia sẻ trọng số đã huấn luyện** rất quan trọng: xây tiếp trên mô hình có sẵn giúp giảm chi phí tính toán cho cả cộng đồng. Có thể ước lượng lượng carbon khi huấn luyện bằng các công cụ như ML CO2 Impact hay Code Carbon.

2.3.3 Học chuyển giao: pretraining và fine-tuning

Tiền huấn luyện (pretraining) là huấn luyện từ số không: trọng số khởi tạo ngẫu nhiên, học trên khối dữ liệu rất lớn, có thể mất nhiều tuần. **Tinh chỉnh** (fine-tuning) là huấn luyện thêm *sau khi* đã có mô hình tiền huấn luyện, dùng bộ dữ liệu nhỏ cho tác vụ của bạn. Quan hệ giữa hai bước xem Hình 2.4.



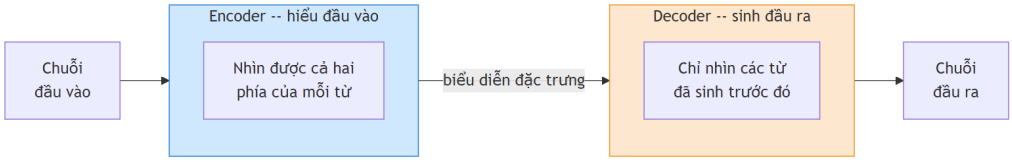
Hình 2.4: Học chuyển giao: kiến thức từ pretraining được "chuyển" sang tác vụ mới.

Vì sao không huấn luyện thẳng cho tác vụ cuối từ đầu? Vì tinh chỉnh tận dụng được kiến thức ngôn ngữ mà mô hình nền đã học, nên cần *ít dữ liệu hơn*, *ít thời gian và tài nguyên hơn*, và thường cho kết quả tốt hơn. Nguyên tắc: luôn ưu tiên lấy một mô hình tiền huấn luyện càng gần tác vụ càng tốt rồi tinh chỉnh.

2.3.4 Kiến trúc tổng quát: Encoder và Decoder

Transformer gồm hai khối chính (Hình 2.5):

- **Encoder** – nhận đầu vào và xây dựng biểu diễn (đặc trưng) của nó. Tối ưu cho việc *hiểu* đầu vào.
- **Decoder** – dùng biểu diễn từ encoder cùng các đầu vào khác để sinh chuỗi đích. Tối ưu cho việc *sinh* đầu ra.



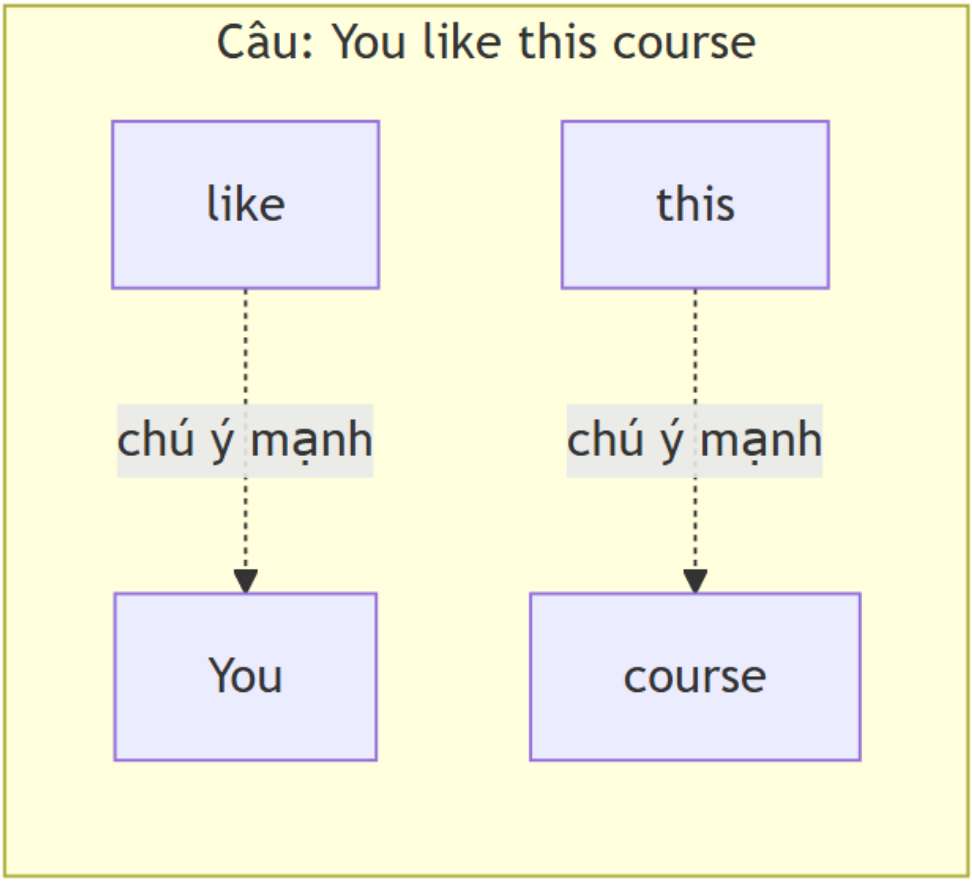
Hình 2.5: Hai khối encoder và decoder cùng vai trò của chúng.

Mỗi khối dùng được độc lập tuỳ tác vụ: **chỉ encoder** hợp với tác vụ cần hiểu đầu vào (phân loại câu, NER); **chỉ decoder** hợp với tác vụ sinh (sinh văn bản); **encoder-decoder** hợp với tác vụ sinh có điều kiện trên đầu vào (dịch, tóm tắt).

2.3.5 Lớp attention: trái tim của Transformer

Đặc trưng cốt lõi của Transformer là **lớp attention** (chú ý). Tên bài báo gốc là "Attention Is All You Need". Ý tưởng: khi xử lý biểu diễn của mỗi từ, lớp này bảo mô hình *chú ý mạnh vào một số từ liên quan* và gần như bỏ qua những từ còn lại.

Lấy ví dụ dịch "You like this course" sang tiếng Pháp (Hình 2.6). Để dịch từ "like", mô hình phải chú ý tới chủ ngữ "You" vì động từ chia khác nhau theo chủ ngữ. Để dịch "this", mô hình phải chú ý tới danh từ "course" vì cách dịch phụ thuộc giống của danh từ. Nói rộng ra: nghĩa của một từ chịu ảnh hưởng sâu sắc bởi ngữ cảnh – những từ khác đứng trước hoặc sau nó.



Hình 2.6: Trực giác về attention: mỗi từ "nhìn" sang các từ liên quan để hiểu đúng.

2.3.6 Kiến trúc gốc và che attention

Transformer gốc thiết kế cho dịch. Khi huấn luyện: encoder nhận câu nguồn, decoder nhận câu đích. Encoder được phép nhìn mọi từ trong câu (vì nghĩa một từ phụ thuộc cả hai phía). Decoder làm việc tuần tự: khi sinh một từ, nó chỉ được nhìn các từ đã sinh trước đó – nếu được nhìn cả từ cần đoán thì bài toán quá dễ. Cơ chế này gọi là **che attention**: đặt điểm chú ý tới các từ "tương lai" về mức vô hiệu.

Trong khối decoder, lớp attention thứ nhất chỉ nhìn các đầu vào quá khứ của decoder, còn lớp thứ hai

dùng đầu ra của encoder – nhờ vậy nó truy cập được toàn bộ câu nguồn để đoán từ hiện tại. Điều này hữu ích vì các ngôn ngữ có trật tự từ khác nhau, hoặc ngữ cảnh ở cuối câu có thể giúp dịch đúng một từ ở đầu. Che attention cũng dùng để mô hình bỏ qua các từ đặc biệt như từ đệm (padding) thêm vào để các câu trong cùng batch dài bằng nhau.

Kiến trúc, checkpoint và mô hình

- **Kiến trúc** (architecture): bộ khung của mô hình – định nghĩa từng lớp và phép toán. Ví dụ: BERT.
- **Checkpoint**: bộ trọng số nạp vào một kiến trúc. Ví dụ: `bert-base-cased`.
- **Mô hình** (model): từ chung, có thể hiểu là kiến trúc hoặc checkpoint tuỳ ngữ cảnh.

2.4 Cách Transformer giải các bài toán

Nhờ kiến trúc linh hoạt, hầu hết mô hình là biến thể của một trong ba dạng: chỉ encoder, chỉ decoder, hoặc encoder-decoder. Mọi tác vụ đều theo một khuôn chung: *dữ liệu vào đi qua mô hình, đầu ra được diễn giải cho tác vụ cụ thể*. Khác biệt nằm ở cách chuẩn bị dữ liệu, chọn biến thể kiến trúc nào, và xử lý đầu ra ra sao. Thường thì: tác vụ cần ngữ cảnh hai chiều dùng encoder; tác vụ sinh văn bản dùng decoder; tác vụ đổi chuỗi này sang chuỗi khác dùng encoder-decoder.

2.4.1 Sinh văn bản với mô hình kiểu GPT

GPT-2 là mô hình **chỉ decoder**. Luồng xử lý ở mức ý tưởng:

1. Văn bản được tách thành token và chuyển thành vector nhúng (embedding); thêm thông tin vị trí để mô hình biết thứ tự các token. Chuỗi này đi qua nhiều khối decoder. Trong mỗi khối có **che tự-attention**: mô hình chỉ được nhìn các token bên trái, không nhìn token tương lai.
2. Đầu ra decoder đi qua một "đầu" mô hình ngôn ngữ để quy đổi trạng thái ẩn thành điểm số cho từng từ trong từ vựng; từ nào điểm cao nhất là từ tiếp theo được chọn.

Mục tiêu tiền huấn luyện của GPT-2 hoàn toàn là *đoán từ kế tiếp*, nên nó đặc biệt giỏi sinh văn bản.

2.4.2 Phân loại, gán nhãn token và hỏi đáp với BERT

BERT là mô hình **chỉ encoder**, mô hình đầu tiên hiện thực hoá hiệu quả tính *hai chiều* sâu – mỗi từ "nhìn" được cả hai phía để tạo biểu diễn giàu hơn.

- **Token đặc biệt**: thêm token `[CLS]` ở đầu mỗi chuỗi; trạng thái ẩn cuối cùng tại `[CLS]` dùng làm đầu vào cho đầu phân loại. Token `[SEP]` để tách hai câu trong một cặp.
- **Hai mục tiêu tiền huấn luyện**: (1) *che từ* – đoán lại các token bị che; (2) *đoán câu kế tiếp* – phán đoán câu B có nối tiếp câu A hay không.

Điểm hay: sau khi đã tiền huấn luyện, dùng BERT cho tác vụ mới rất gọn – chỉ cần gắn thêm một "đầu" phù hợp lên trên mô hình nền:

- **Phân loại văn bản**: gắn đầu phân loại chuỗi lên trạng thái ẩn của `[CLS]`.
- **Gán nhãn token** (NER, từ loại): gắn đầu phân loại token để gán nhãn cho từng token.
- **Hỏi đáp**: gắn đầu dự đoán khoảng (span) – xác định vị trí bắt đầu và kết thúc của câu trả lời trong ngữ cảnh.

2.4.3 Tóm tắt và dịch với encoder-decoder

Tóm tắt và dịch là tác vụ chuỗi-sang-chuỗi, hợp với mô hình encoder-decoder như BART và T5. Với BART, ý tưởng tiền huấn luyện là *làm hỏng đầu vào rồi dựng lại*: ví dụ thay một số đoạn văn bản bằng một token che duy nhất, encoder đọc phần bị làm hỏng, decoder phải khôi phục văn bản gốc. Nhờ vừa hiểu (encoder) vừa sinh (decoder), kiến trúc này rất hợp để biến một chuỗi đầu vào thành một chuỗi đầu ra khác.

2.4.4 Vượt ra ngoài văn bản

Transformer không chỉ dùng cho văn bản. Vài ví dụ tiêu biểu:

- **Tiếng nói**: Whisper là encoder-decoder huấn luyện trên lượng lớn âm thanh có nhãn, chạy tốt kiểu zero-shot cho nhận dạng tiếng nói nhiều ngôn ngữ. Âm thanh thô được đổi thành phổ (spectrogram) trước khi vào encoder; decoder sinh dần các token văn bản.
- **Hình ảnh**: ViT (Vision Transformer) cắt ảnh thành các mảnh vuông không chồng nhau, biến mỗi mảnh thành một vector – giống như "token hoá" ảnh. Thêm token `[CLS]` và thông tin vị trí, rồi đưa qua encoder; đầu ra tại `[CLS]` dùng để phân loại ảnh. Hãy để ý sự tương đồng với BERT.

Nhiều mô hình tuy giải tác vụ khác nhau nhưng theo cùng khuôn mẫu. Nắm được các khuôn chung này giúp bạn nhanh chóng hiểu mô hình mới và biết cách điều chỉnh mô hình có sẵn cho nhu cầu của mình.

Tóm tắt chương

- LLM là mô hình ngôn ngữ quy mô lớn, học tự giám sát trên văn bản khổng lồ; mạnh nhưng có giới hạn (ảo giác, thiên lệch, cửa sổ ngữ cảnh, chi phí).
- `pipeline` là cách nhanh nhất để thử các tác vụ: phân loại, sinh văn bản, NER, hỏi đáp, tóm tắt, dịch…
- Transformer gồm encoder (hiểu) và decoder (sinh); attention giúp mỗi từ chú ý vào ngữ cảnh liên quan.
- Ba họ mô hình – chỉ encoder, chỉ decoder, encoder-decoder – ứng với ba nhóm tác vụ khác nhau.
- Học chuyển giao (pretraining rồi fine-tuning) là mô thức chủ đạo và tiết kiệm.

Chương 3

Dùng thư viện Transformers

Chương 2 dùng `pipeline` như một hộp đen. Chương này mở hộp đó ra: xem từng bước bên trong, rồi học cách dùng trực tiếp **tokenizer** và **mô hình** để kiểm soát nhiều hơn. Toàn bộ ví dụ dùng PyTorch làm backend.

3.1 Bên trong một pipeline

Nhớ lại ví dụ phân tích cảm xúc ở chương trước. Bên trong, `pipeline` gom ba bước: tiền xử lý bằng tokenizer, đưa qua mô hình, rồi hậu xử lý (Hình 3.1).



Hình 3.1: Luồng đầy đủ: từ văn bản thô tới nhãn và điểm tin cậy.

3.1.1 Bước 1 – Tiền xử lý bằng tokenizer

Như mọi mạng nơ-ron, Transformer không đọc được văn bản thô, nên bước đầu là biến văn bản thành số. Việc này do tokenizer đảm nhiệm: tách đầu vào thành token (từ, từ phụ, hay ký hiệu), ánh xạ mỗi token sang một số nguyên, và thêm vài đầu vào phụ trợ hữu ích. Toàn bộ phải làm *giống hệt* lúc mô hình được tiền huấn luyện, nên ta tải đúng thông tin đó qua lớp `AutoTokenizer`.

```
from transformers import AutoTokenizer

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

raw_inputs = [
    "I've been waiting for a HuggingFace course my whole life.",
    "I hate this so much!",
]
inputs = tokenizer(raw_inputs, padding=True, truncation=True, return_tensors="pt")
print(inputs)
```

Kết quả là một từ điển có hai khoá chính: `input_ids` (mỗi hàng là dãy số nguyên định danh token của một câu) và `attention_mask` (sẽ giải thích ở phần sau). Tham số `return_tensors="pt"` yêu cầu trả về tensor PyTorch – Transformer chỉ nhận tensor làm đầu vào. Có thể hình dung tensor như mảng NumPy nhiều chiều.

3.1.2 Bước 2 – Đưa qua mô hình

Tải mô hình giống như tải tokenizer, qua lớp `AutoModel`. Lớp này chỉ chứa phần Transformer nền: cho một đầu vào, nó xuất ra **trạng thái ẩn** (hidden states), còn gọi là *đặc trưng* – mỗi token được biểu diễn bằng một vector nhiều chiều mang hiểu biết theo ngữ cảnh.

```
from transformers import AutoModel

model = AutoModel.from_pretrained(checkpoint)
outputs = model(**inputs)
print(outputs.last_hidden_state.shape)
# torch.Size([2, 16, 768])
```

Vector đầu ra có ba chiều: **kích thước batch** (số câu xử lý cùng lúc, ở đây là 2), **độ dài chuỗi** (số token, ở đây 16), và **kích thước ẩn** (số chiều của mỗi vector token, thường 768 với mô hình nhỏ, có thể tới 3072 hoặc hơn ở mô hình lớn).

3.1.2.1 Đầu mô hình: biến con số thành kết quả

Trạng thái ẩn thường được đưa tiếp vào một **đầu** (head) – vài lớp tuyến tính chiếu vector nhiều chiều về chiều khác phù hợp với tác vụ. Mỗi tác vụ dùng một đầu khác nhau. Thư viện có sẵn nhiều kiến trúc kèm đầu:

Lớp	Dùng cho
`*Model`	lấy trạng thái ẩn (không có đầu tác vụ).
`*ForCausalLM`	sinh văn bản (đoán từ kế tiếp).
`*ForMaskedLM`	điền token bị che.
`*ForQuestionAnswering`	hỏi đáp trích đoạn.
`*ForSequenceClassification`	phân loại cả câu.
`*ForTokenClassification`	gán nhãn từng token (NER, từ loại).

Bảng 3.1: Một số lớp mô hình kèm đầu tác vụ.

Với phân loại cảm xúc, ta dùng `AutoModelForSequenceClassification`:

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
outputs = model(**inputs)
print(outputs.logits.shape)
# torch.Size([2, 2])  -- 2 câu x 2 nhãn
```

3.1.3 Bước 3 – Hậu xử lý

Đầu ra của mô hình là **logits** – điểm thô, chưa chuẩn hoá, do lớp cuối xuất ra. Chúng chưa phải xác suất. Để thành xác suất, cho qua một lớp chuẩn hoá (hàm softmax): nó biến dãy điểm thô thành các số dương cộng lại bằng 1, giữ nguyên thứ tự lớn nhỏ.

```
import torch

print(outputs.logits)
# tensor([[-1.5607,  1.6123], [ 4.1692, -3.3464]])
predictions = torch.nn.functional.softmax(outputs.logits, dim=-1)
```

```
print(predictions)
# tensor([[0.0402, 0.9598], [0.9995, 0.0005]])
print(model.config.id2label)
# {0: 'NEGATIVE', 1: 'POSITIVE'}
```

Vậy câu một: NEGATIVE 0.0402, POSITIVE 0.9598; câu hai: NEGATIVE 0.9995, POSITIVE 0.0005 – khớp đúng kết quả của `pipeline` ở chương trước.

3.2 Mô hình: tạo, lưu, chia sẻ

Phương thức `from_pretrained()` tải và cache mô hình từ Hub. Tên checkpoint ứng với một kiến trúc và bộ trọng số cụ thể; ví dụ `bert-base-cased` là BERT 12 lớp, kích thước ẩn 768, phân biệt hoa thường.

```
from transformers import AutoModel
model = AutoModel.from_pretrained("bert-base-cased")
```

Lớp `AutoModel` chỉ là "lớp bọc" tự đoán đúng kiến trúc theo checkpoint. Nếu đã biết loại mô hình, có thể gọi thẳng lớp của nó (ví dụ `BertModel`).

Lưu mô hình bằng `save_pretrained("thu_muc")`, sinh ra hai tệp: `config.json` (mô tả kiến trúc và metadata) và tệp trọng số `.safetensors` (toàn bộ tham số). Hai tệp bổ trợ nhau: cấu hình để biết kiến trúc, trọng số là tham số đã học.

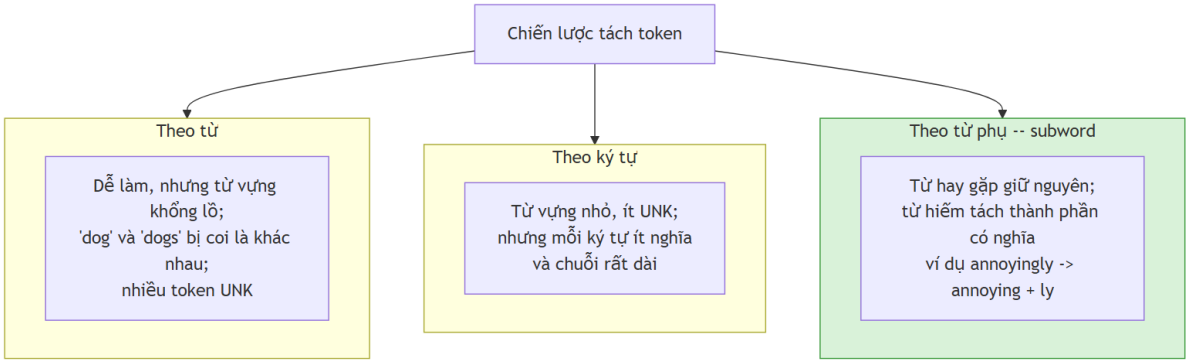
Chia sẻ lên Hub rất gọn: đăng nhập rồi gọi `push_to_hub`.

```
from huggingface_hub import notebook_login
notebook_login()                  # hoặc: huggingface-cli login

model.push_to_hub("my-awesome-model")
# Người khác tải lại:
# AutoModel.from_pretrained("your-username/my-awesome-model")
```

3.3 Tokenizer: biến văn bản thành số

Tokenizer dịch văn bản thành dữ liệu mô hình xử lý được. Có ba chiến lược tách token chính (Hình 3.2).



Hình 3.2: Ba chiến lược tách token và đánh đổi của mỗi loại.

Theo từ dễ làm nhưng từ vựng khổng lồ (tiếng Anh hơn 500.000 từ), và "dog" với "dogs" bị xem là không liên quan; từ lạ rơi vào token "không biết" (`[UNK]`), làm mất thông tin. **Theo ký tự** có từ vựng

nhỏ, ít [UNK], nhưng mỗi ký tự ít nghĩa và chuỗi rất dài. **Theo từ phụ** (subword) kết hợp ưu điểm: từ hay gặp giữ nguyên, từ hiếm tách thành phần có nghĩa – ví dụ "annoyingly" thành "annoying" + "ly". Các biến thể phổ biến: byte-level BPE (GPT-2), WordPiece (BERT), SentencePiece / Unigram (nhiều mô hình đa ngôn ngữ).

3.3.1 Mã hoá và giải mã

Mã hoá gồm hai bước: tách token rồi đổi token sang ID.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

sequence = "Using a Transformer network is simple"
tokens = tokenizer.tokenize(sequence)
print(tokens)
# ['Using', 'a', 'transform', '##er', 'network', 'is', 'simple']

ids = tokenizer.convert_tokens_to_ids(tokens)
print(ids)
# [7993, 170, 11303, 1200, 2443, 1110, 3014]
```

Để ý "transformer" bị tách thành "transform" và "##er" – dấu hiệu đây là tokenizer theo từ phụ. **Giải mã** đi ngược lại, và còn gộp các mảnh cùng một từ thành câu đọc được:

```
tokenizer.decode([7993, 170, 11303, 1200, 2443, 1110, 3014])
# 'Using a Transformer network is simple'
```

3.3.2 Token đặc biệt

Khi gọi tokenizer trực tiếp (thay vì tách thủ công), nó tự thêm token đặc biệt mà mô hình mong đợi – ví dụ BERT thêm [CLS] ở đầu và [SEP] ở cuối.

```
encoded = tokenizer("How are you?")
tokenizer.decode(encoded["input_ids"])
# '[CLS] How are you? [SEP]'
```

Không phải mô hình nào cũng thêm token đặc biệt, hoặc thêm loại khác; tokenizer biết đúng cái cần và lo giúp bạn.

3.4 Xử lý nhiều câu một lúc

3.4.1 Mô hình mong đợi một batch

Nếu đưa một câu đã mã hoá thành danh sách ID phẳng (một chiều), mô hình sẽ báo lỗi, vì mặc định nó mong đợi *nhiều* câu. Tokenizer khi gọi trực tiếp đã tự thêm một chiều "batch" bên ngoài. Tự tay thì cần bọc danh sách ID trong một danh sách nữa:

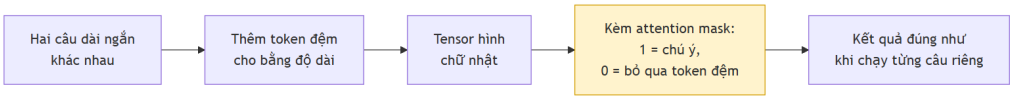
```
import torch

input_ids = torch.tensor([ids])   # thêm chiều batch
output = model(input_ids)
```

Batching là đưa nhiều câu qua mô hình cùng lúc. Một câu cũng có thể gói thành batch một phần tử.

3.4.2 Đệm (padding) và attention mask

Các câu trong batch thường dài ngắn khác nhau, nhưng tensor phải hình chữ nhật. Giải pháp: **đệm** thêm token đặc biệt vào câu ngắn cho bằng câu dài nhất (Hình 3.3). ID của token đệm nằm ở thuộc tính `pad_token_id` của tokenizer.



Hình 3.3: Đệm làm tensor vuông vức; attention mask bảo mô hình bỏ qua token đệm.

Nhưng chỉ đệm thôi thì sai kết quả: lớp attention vẫn "chú ý" cả token đệm, làm thay đổi biểu diễn các câu khác. Cần **attention mask** – tensor cùng hình với input, chứa 1 (chú ý) và 0 (bỏ qua).

```
batched_ids = [
    [200, 200, 200],
    [200, 200, tokenizer.pad_token_id],
]
attention_mask = [
    [1, 1, 1],
    [1, 1, 0],
]
outputs = model(torch.tensor(batched_ids),
                attention_mask=torch.tensor(attention_mask))
```

Có attention mask, câu thứ hai trong batch cho logits đúng bằng khi chạy riêng.

3.4.3 Chuỗi quá dài

Mỗi mô hình có giới hạn độ dài (thường 512 hoặc 1024 token) và sẽ lỗi nếu vượt quá. Hai cách xử lý: dùng mô hình hỗ trợ chuỗi dài hơn (ví dụ Longformer, LED), hoặc cắt bớt chuỗi bằng tham số `truncation`.

3.5 Gộp tất cả: tokenizer làm hết

Thực tế, không cần làm thủ công: gọi tokenizer trực tiếp là nó lo đệm, cắt, thêm token đặc biệt và trả tensor đúng framework. Đây là cách nên dùng trong dự án.

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

sequences = ["I've been waiting for a HuggingFace course my whole life.",
             "So have I!"]
tokens = tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")
output = model(**tokens)
```

Vài lựa chọn hữu ích: `padding="longest"` (đệm tới câu dài nhất), `padding="max_length"` (đệm tới độ dài tối đa của mô hình), `return_tensors="pt"` hoặc `"np"` (PyTorch hay NumPy).

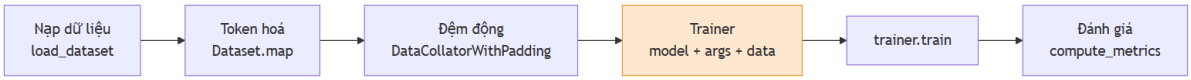
Tóm tắt chương

- `pipeline` = tokenizer (tiền xử lý) + mô hình + hậu xử lý.
- `AutoTokenizer` và `AutoModel*` tự chọn đúng lớp theo checkpoint; luôn dùng cùng checkpoint cho cả hai.
- Mô hình xuất logits (điểm thô); dùng softmax để ra xác suất.
- Batch nhiều câu cần đệm + attention mask để kết quả không bị token đệm làm lệch.
- Trong dự án, hãy gọi tokenizer trực tiếp với `padding`, `truncation`, `return_tensors` thay vì làm từng bước thủ công.

Chương 4

Tinh chỉnh mô hình đã huấn luyện

Chương này đi qua trọn một quy trình **fine-tuning**: nạp dữ liệu, chuẩn bị dữ liệu cho mô hình, rồi huấn luyện bằng lớp `Trainer` cao cấp. Toàn bộ luồng tóm tắt ở Hình 4.1.



Hình 4.1: Quy trình tinh chỉnh một mô hình phân loại.

4.1 Huấn luyện trên một batch

Trước khi dùng dữ liệu lớn, hãy xem một bước huấn luyện tối giản để nắm "bộ xương": chuẩn bị đầu vào kèm nhãn, tính độ lệch (loss), lan truyền ngược, rồi cập nhật tham số.

```
import torch
from torch.optim import AdamW
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

sequences = ["I've been waiting for a HuggingFace course my whole life.",
             "This course is amazing!"]
batch = tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")
batch["labels"] = torch.tensor([1, 1])           # thêm nhãn

optimizer = AdamW(model.parameters())
loss = model(**batch).loss                        # mô hình tự tính loss khi có nhãn
loss.backward()                                   # lan truyền ngược
optimizer.step()                                  # cập nhật tham số
```

Dĩ nhiên huấn luyện trên hai câu không cho kết quả tốt. Để tốt hơn, cần bộ dữ liệu lớn.

4.2 Chuẩn bị dữ liệu

Ví dụ dùng bộ **MRPC** (một trong 10 bộ của benchmark GLUE): 5801 cặp câu, mỗi cặp có nhãn cho biết hai câu có cùng nghĩa (paraphrase) hay không. Bộ nhỏ nên dễ thử nghiệm.

4.2.1 Nạp dữ liệu từ Hub

Thư viện `datasets` tải và cache dữ liệu bằng một lệnh:

```
from datasets import load_dataset

raw_datasets = load_dataset("glue", "mrpc")
# DatasetDict với 3 phần: train (3668), validation (408), test (1725)
# mỗi phần có cột: sentence1, sentence2, label, idx
```

Nhãn đã là số nguyên. Xem thuộc tính `features` để biết số nào ứng với nhãn nào: số `0` nghĩa là hai câu khác nghĩa, số `1` nghĩa là hai câu cùng nghĩa.

4.2.2 Token hoá cặp câu

Tokenizer nhận được *cặp* câu và chuẩn bị đúng dạng BERT mong đợi: `[CLS] câu1 [SEP] câu2 [SEP]`.

```
inputs = tokenizer("This is the first sentence.", "This is the second one.")
tokenizer.convert_ids_to_tokens(inputs["input_ids"])
# ['[CLS]','this','is','the','first','sentence','.','[SEP]',
#  'this','is','the','second','one','.','[SEP]']
```

Ở đây xuất hiện khoá mới `token_type_ids`: nó cho mô hình biết phần nào là câu thứ nhất (gán 0), phần nào là câu thứ hai (gán 1). BERT dùng thông tin này vì ngoài mục tiêu che từ, nó còn được tiền huấn luyện với tác vụ *đoán câu kế tiếp*. Nếu đổi sang checkpoint khác (ví dụ DistilBERT) có thể không có `token_type_ids` – không sao, miễn tokenizer và mô hình dùng cùng checkpoint.

4.2.3 Áp token hoá cho cả tập với `map`

Thay vì token hoá tất cả một lần (tốn RAM), ta định nghĩa một hàm và áp bằng `Dataset.map` với `batched=True` để chạy nhanh. Chưa đệm ở bước này – để dành đệm theo từng batch.

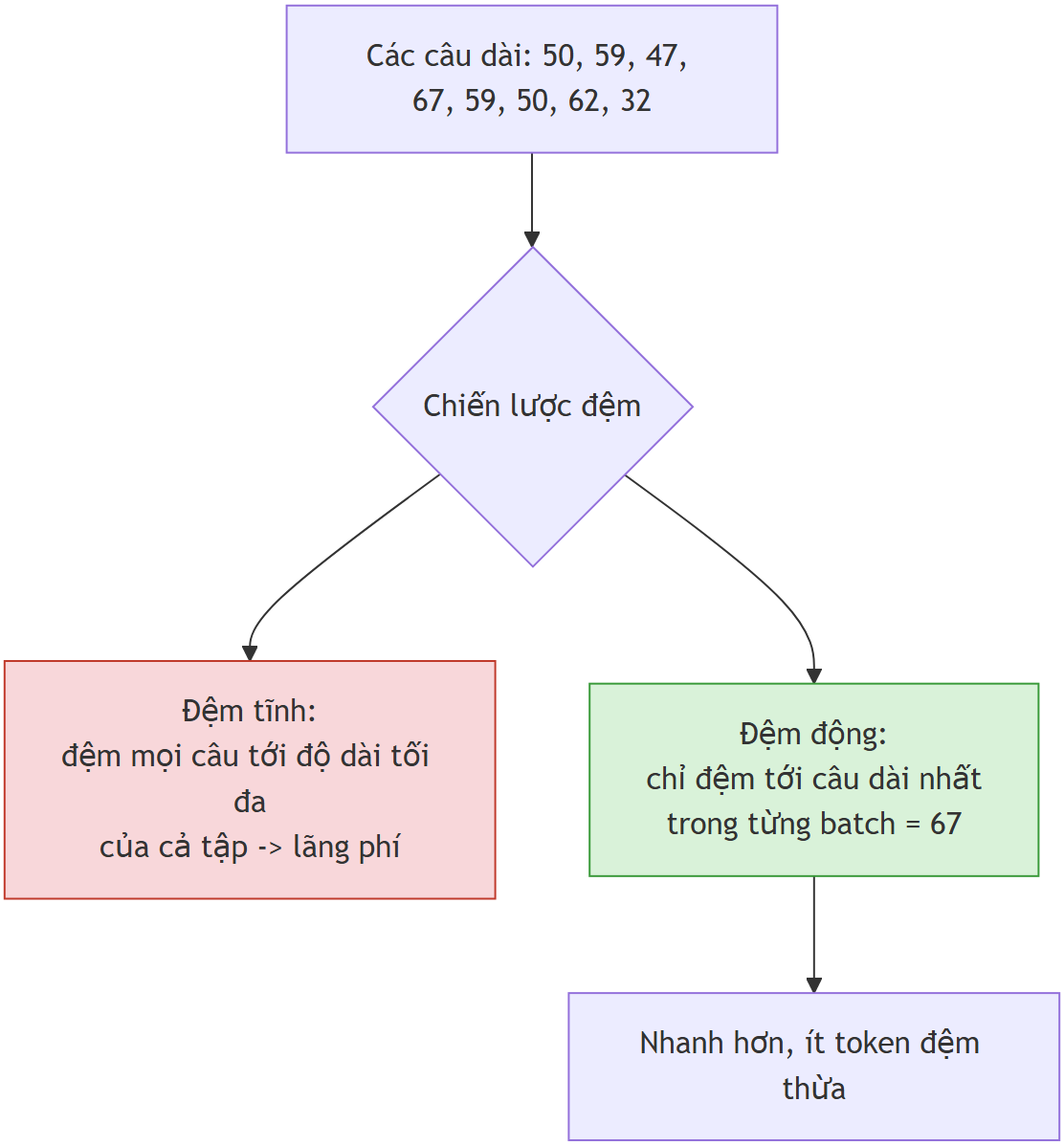
```
def tokenize_function(example):
    return tokenizer(example["sentence1"], example["sentence2"], truncation=True)

tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
```

Dữ liệu được lưu dưới dạng tệp trên đĩa (Apache Arrow), nên chỉ phần cần dùng mới nạp vào bộ nhớ – xử lý được tập rất lớn.

4.2.4 Đệm động

Đệm động (dynamic padding): chỉ đệm các câu trong một batch tới độ dài câu dài nhất *của batch đó*, thay vì tới độ dài tối đa của cả tập (Hình 4.2). Việc này tiết kiệm thời gian và tính toán. Lớp `DataCollatorWithPadding` lo việc này khi gom batch.



Hình 4.2: Đệm động chỉ đệm trong phạm vi batch, tránh token đệm thừa.

```
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
# Ví dụ 8 câu dài [50, 59, 47, 67, 59, 50, 62, 32]
# -> sau collate, tất cả cùng dài 67 (câu dài nhất trong batch)
```

Mẹo

Dùng `batched=True` với `map` cho tốc độ cao, và đệm động với `DataCollatorWithPadding` thay vì đệm cố định. Nếu huấn luyện trên TPU thì ngược lại – TPU thích hình dạng cố định nên đệm cố định có thể phù hợp hơn.

4.3 Huấn luyện với `Trainer`

Lớp `Trainer` đóng gói các thực hành tốt hiện đại, giúp tinh chỉnh mô hình với rất ít dòng lệnh. Lưu ý: chạy trên CPU sẽ rất chậm – nên dùng GPU (ví dụ GPU miễn phí trên Google Colab).

4.3.1 Khai báo tham số và mô hình

`TrainingArguments` chứa mọi siêu tham số. Bắt buộc chỉ cần thư mục lưu kết quả; còn lại dùng mặc định là chạy ổn cho fine-tuning cơ bản.

```
from transformers import TrainingArguments, AutoModelForSequenceClassification

training_args = TrainingArguments("test-trainer")
model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
```

Khi tạo mô hình sẽ có cảnh báo: vì BERT chưa từng được huấn luyện để phân loại cặp câu, đầu tiền huấn luyện bị bỏ đi và một đầu phân loại mới được gắn vào với trọng số khởi tạo ngẫu nhiên. Cảnh báo này là bình thường – nó nhắc rằng bạn cần huấn luyện mô hình, đúng việc ta sắp làm.

4.3.2 Tạo `Trainer` và huấn luyện

```
from transformers import Trainer

trainer = Trainer(
    model,
    training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    processing_class=tokenizer,     # báo Trainer dùng tokenizer nào
)
trainer.train()
```

Khi đưa tokenizer qua `processing_class`, `Trainer` tự dùng `DataCollatorWithPadding` làm collator mặc định (nên dòng `data_collator` có thể bỏ). Lệnh `train()` bắt đầu tinh chỉnh và báo loss huấn luyện mỗi 500 bước.

4.3.3 Đánh giá

Mặc định `train()` chưa cho biết mô hình tốt hay dở, vì ta chưa bật đánh giá và chưa cung cấp hàm tính độ đo. Lấy dự đoán bằng `trainer.predict`; đầu ra là logits nên cần lấy chỉ số có giá trị lớn nhất để ra nhãn dự đoán.

```
import numpy as np
import evaluate

def compute_metrics(eval_preds):
    metric = evaluate.load("glue", "mrpc")
    logits, labels = eval_preds
    predictions = np.argmax(logits, axis=-1)   # chọn nhãn điểm cao nhất
    return metric.compute(predictions=predictions, references=labels)
```

Gắn hàm này vào `Trainer` và bật đánh giá cuối mỗi epoch bằng `eval_strategy="epoch"`:

```
training_args = TrainingArguments("test-trainer", eval_strategy="epoch")
model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)

trainer = Trainer(
    model, training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    processing_class=tokenizer,
    compute_metrics=compute_metrics,
)
trainer.train()
```

Trên tập validation của MRPC, tài liệu gốc đạt độ chính xác khoảng **85,78%** và điểm F1 khoảng **89,97**. Con số có thể dao động nhẹ giữa các lần chạy do đầu phân loại được khởi tạo ngẫu nhiên.

4.3.4 Vài tính năng huấn luyện nâng cao

`Trainer` có sẵn nhiều lựa chọn tối ưu, bật bằng tham số trong `TrainingArguments`:

- **Độ chính xác hỗn hợp** (`fp16=True`): huấn luyện nhanh hơn, tốn ít bộ nhớ hơn.
- **Tích luỹ gradient** (`gradient_accumulation_steps`): mô phỏng batch lớn khi GPU ít bộ nhớ – ví dụ batch 4 nhân 4 bước bằng batch hiệu dụng 16.
- **Lịch học** (`lr_scheduler_type`): mặc định giảm tuyến tính, có thể đổi sang kiểu khác như `cosine`.

`Trainer` cũng chạy được trên nhiều GPU/TPU và hỗ trợ huấn luyện phân tán.

4.4 Vòng lặp huấn luyện thủ công

Khi cần kiểm soát sâu, có thể tự viết vòng lặp bằng PyTorch thay cho `Trainer`. Ý tưởng y hệt bước "một batch" ở đầu chương, chỉ khác là lặp qua toàn bộ dữ liệu nhiều epoch: với mỗi batch, tính loss, gọi lan truyền ngược, cập nhật tham số bằng optimizer, cập nhật lịch học, rồi xoá gradient cho bước sau. Thư viện **Accelerate** giúp cùng một đoạn code chạy được trên nhiều GPU/TPU hoặc máy phân tán mà gần như không phải sửa gì.

Tóm tắt chương

- Fine-tuning gồm: nạp dữ liệu, token hoá bằng `map(batched=True)`, đệm động, rồi huấn luyện.
- `token_type_ids` phân biệt câu 1 và câu 2 khi đầu vào là cặp câu (BERT).
- `Trainer` là cách nhanh và gọn nhất; chỉ cần `TrainingArguments`, mô hình, dữ liệu, và (tuỳ chọn) `compute_metrics`.
- Bật `eval_strategy="epoch"` để theo dõi độ đo trong lúc huấn luyện.
- Cần kiểm soát sâu thì viết vòng lặp PyTorch thủ công và dùng Accelerate để mở rộng.

Chương 5

Chia sẻ mô hình và tokenizer

Model Hub là kho chứa hàng triệu mô hình tiền huấn luyện, giúp chọn và dùng mô hình chỉ với vài dòng lệnh. Chương này xem cách *dùng* mô hình có sẵn và cách *đóng góp* ngược lại cho cộng đồng.

5.1 Dùng mô hình tiền huấn luyện

Chỉ cần tên checkpoint là đủ để dùng mô hình trong `pipeline`. Ví dụ một mô hình tiếng Pháp cho tác vụ điền chỗ trống:

```
from transformers import pipeline

camembert_fill_mask = pipeline("fill-mask", model="camembert-base")
results = camembert_fill_mask("Le camembert est <mask> :)")
# 'délicieux' (0.4909), 'excellent' (0.1056), ...
```

Điều duy nhất cần để ý: checkpoint phải *hợp* với tác vụ. Nạp `camembert-base` vào `fill-mask` thì đúng, nhưng nạp vào `text-classification` thì vô nghĩa vì đầu của nó không hợp tác vụ đó. Nên dùng bộ lọc tác vụ trên giao diện Hub để chọn checkpoint phù hợp.

Khuyến nghị dùng các lớp `Auto*` vì chúng không phụ thuộc kiến trúc – đổi checkpoint rất dễ:

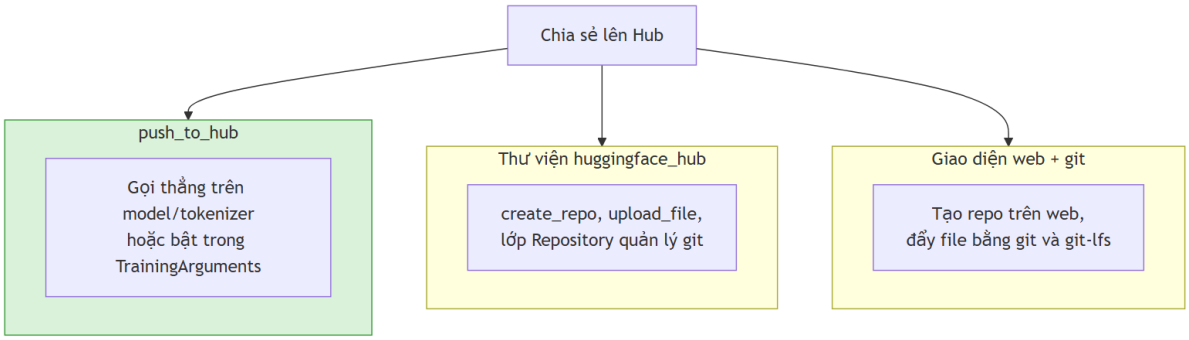
```
from transformers import AutoTokenizer, AutoModelForMaskedLM

tokenizer = AutoTokenizer.from_pretrained("camembert-base")
model = AutoModelForMaskedLM.from_pretrained("camembert-base")
```

> **Lưu ý**
>
> Khi dùng một mô hình tiền huấn luyện, hãy kiểm tra nó được huấn luyện thế nào, trên dữ liệu nào, giới hạn và thiên lệch ra sao. Mọi thông tin này nên có trong *thẻ mô hình* (model card).

5.2 Chia sẻ mô hình lên Hub

Có ba cách tạo kho mô hình mới (Hình 5.1): dùng API `push_to_hub`, dùng thư viện `huggingface_hub`, hoặc dùng giao diện web.



Hình 5.1: Ba cách đưa mô hình lên Hub.

Trước tiên cần đăng nhập để có token xác thực:

```
from huggingface_hub import notebook_login
notebook_login()              # hoặc trong terminal: huggingface-cli login
```

5.2.1 Cách 1 – API `push_to_hub`

Nếu dùng `Trainer`, chỉ cần bật `push_to_hub=True` trong `TrainingArguments`; mỗi lần lưu, mô hình sẽ được đẩy lên kho mang tên thư mục đầu ra. Kết thúc huấn luyện, gọi `trainer.push_to_hub()` để đẩy bản cuối kèm thẻ mô hình tự sinh (ghi lại siêu tham số và kết quả đánh giá).

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    "bert-finetuned-mrpc", save_strategy="epoch", push_to_hub=True
)
```

Ở mức thấp hơn, mọi `model`, `tokenizer` đều có sẵn phương thức `push_to_hub`, tự lo tạo kho và đẩy file:

```
model.push_to_hub("dummy-model")
tokenizer.push_to_hub("dummy-model")
# tải lên namespace của tổ chức:
# tokenizer.push_to_hub("dummy-model", organization="huggingface")
```

5.2.2 Cách 2 – Thư viện `huggingface_hub`

Thư viện này cung cấp API trên nền git để quản lý kho: tạo/xoá kho, liệt kê, tải file.

```
from huggingface_hub import create_repo

create_repo("dummy-model")
# tham số hữu ích: private (ẩn/công khai), token, repo_type ("dataset"/"space")
```

Để tải file không cần git, dùng `upload_file` (giới hạn tệp dưới 5GB):

```
from huggingface_hub import upload_file

upload_file(
    "<path_to_file>/config.json",
    path_in_repo="config.json",
    repo_id="<namespace>/dummy-model",
)
```

5.2.3 Cách 3 – Giao diện web và git

Tạo kho tại `huggingface.co/new`, chọn chủ sở hữu, đặt tên, chọn công khai hay riêng tư. Hệ thống file dựa trên **git** cho tệp thường và **git-lfs** cho tệp lớn. Quy trình giống git thông thường:

```
git lfs install
git clone https://huggingface.co/<namespace>/<your-model-id>
# lưu model + tokenizer vào thư mục, rồi:
git add .
git commit -m "First model version"
git push
```

File trọng số lớn (ví dụ hơn 400 MB) được git-lfs theo dõi tự động nhờ cấu hình `.gitattributes` có sẵn khi tạo kho.

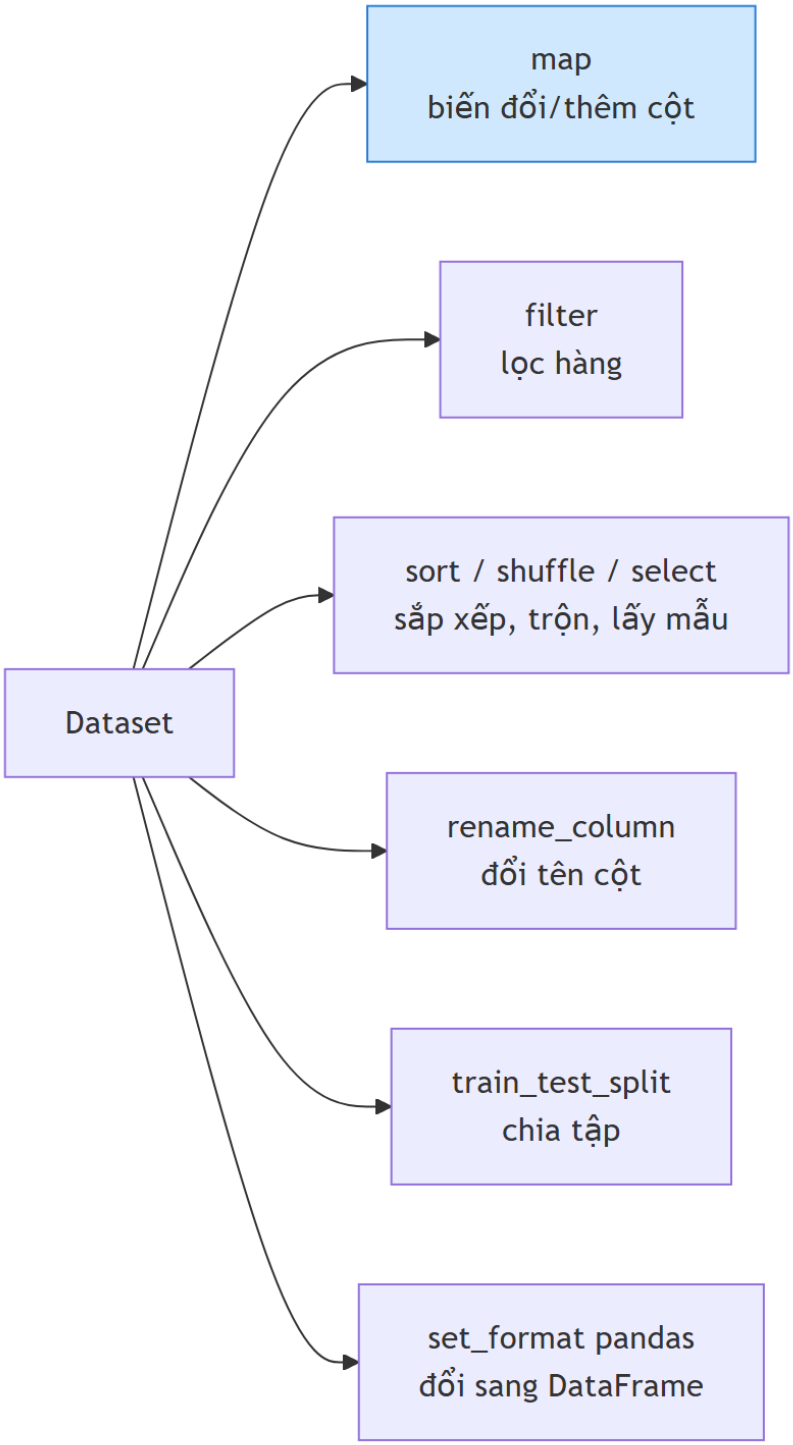
Tóm tắt chương

- Dùng mô hình có sẵn chỉ cần tên checkpoint; ưu tiên lớp `Auto*`.
- Luôn kiểm tra thẻ mô hình để biết dữ liệu huấn luyện, giới hạn, thiên lệch.
- Ba cách chia sẻ: `push_to_hub`, thư viện `huggingface_hub`, và giao diện web kèm git/git-lfs.
- `Trainer` có thể tự đẩy mô hình và sinh thẻ mô hình trong lúc huấn luyện.

Chương 6

Thư viện Datasets

Dữ liệu thực tế hiếm khi sạch sẵn. Thư viện `datasets` cung cấp bộ công cụ nạp, làm sạch, biến đổi và xử lý dữ liệu ở mọi quy mô – từ vài nghìn dòng đến hàng trăm gigabyte. Chương này đi qua các thao tác hay dùng (Hình 6.1) và cách làm việc với dữ liệu lớn.



Hình 6.1: Các thao tác chính trên một `Dataset`.

6.1 Cắt gọt và làm sạch dữ liệu

Ví dụ dùng bộ **Drug Review** (nhận xét của bệnh nhân về thuốc, kèm bệnh và điểm hài lòng 1–10) ở định dạng TSV. Nạp bằng trình đọc CSV với dấu phân tách là tab:

```
from datasets import load_dataset
```

```
data_files = {"train": "drugsComTrain_raw.tsv", "test": "drugsComTest_raw.tsv"}
drug_dataset = load_dataset("csv", data_files=data_files, delimiter="\t")
```

Lấy mẫu ngẫu nhiên để cảm nhận dữ liệu, bằng cách nối `shuffle` và `select`:

```
drug_sample = drug_dataset["train"].shuffle(seed=42).select(range(1000))
drug_sample[:3]
```

Nhìn qua mẫu, phát hiện vài vấn đề: cột `Unnamed: 0` có vẻ là mã bệnh nhân; cột `condition` lẫn hoa thường; nhận xét lẫn ký tự xuống dòng và mã HTML.

6.1.1 Đổi tên cột và chuẩn hoá nhãn

`rename_column` đổi tên trên cả `DatasetDict`. `map` áp một hàm lên mọi hàng; `filter` lọc hàng theo điều kiện.

```
drug_dataset = drug_dataset.rename_column("Unnamed: 0", "patient_id")

# Bỏ các hàng có condition = None rồi mới hạ chữ thường
drug_dataset = drug_dataset.filter(lambda x: x["condition"] is not None)
drug_dataset = drug_dataset.map(lambda x: {"condition": x["condition"].lower()})
```

Hàm lambda

`lambda` là cách định nghĩa hàm nhỏ, dùng một lần, không cần đặt tên. Dạng chung: `lambda tham_số : biểu_thức`. Rất tiện cho các thao tác `map` và `filter` đơn giản.

6.1.2 Tạo cột mới

Nếu hàm trả về khoá chưa có trong dữ liệu, `map` sẽ *thêm cột mới*. Ví dụ đếm số từ mỗi nhận xét rồi lọc bỏ nhận xét quá ngắn:

```
def compute_review_length(example):
    return {"review_length": len(example["review"].split())}

drug_dataset = drug_dataset.map(compute_review_length)
drug_dataset = drug_dataset.filter(lambda x: x["review_length"] > 30)
# bỏ khoảng 15% nhận xét quá ngắn
```

Xử lý mã HTML bằng mô-đun `html` của Python:

```
import html
drug_dataset = drug_dataset.map(lambda x: {"review": html.unescape(x["review"])})
```

6.2 Sức mạnh của `map(batched=True)`

Khi đặt `batched=True`, `map` gửi *cả một lô* ví dụ vào hàm cùng lúc (mặc định 1000). Hàm nhận từ điển mà mỗi giá trị là một *danh sách*, và cũng trả về danh sách. Cách này nhanh hơn nhiều nhờ xử lý theo lô.

```
new_drug_dataset = drug_dataset.map(
    lambda x: {"review": [html.unescape(o) for o in x["review"]]}, batched=True
)
```

`batched=True` đặc biệt quan trọng để khai thác tốc độ của **tokenizer nhanh** (chương sau). Theo đo đạc trong tài liệu gốc, tokenizer nhanh với `batched=True` nhanh gấp khoảng **30 lần** so với tokenizer chậm không gom lô. Lý do: phần token hoá chạy bằng Rust nên dễ song song hoá. Thao tác `map` cũng tự song song hoá được qua tham số `num_proc` (hữu ích khi tokenizer không có bản nhanh).

> **Lưu ý**
>
> Nói chung không nên dùng đa tiến trình (`num_proc`) của Python cho tokenizer nhanh khi đã bật `batched=True` – bản thân nó đã song song hoá rồi.

6.2.1 Một ví dụ sinh ra nhiều đặc trưng

Với `batched=True`, `map` còn thay đổi được *số lượng* phần tử. Ví dụ cắt mỗi nhận xét dài thành nhiều đoạn 128 token bằng `return_overflowing_tokens=True`. Khi đó số đặc trưng mới nhiều hơn số ví dụ gốc, nên cần bỏ các cột cũ (`remove_columns`) hoặc nhân bản chúng cho khớp độ dài.

```
def tokenize_and_split(examples):
    return tokenizer(
        examples["review"], truncation=True, max_length=128,
        return_overflowing_tokens=True,
    )

tokenized = drug_dataset.map(
    tokenize_and_split, batched=True,
    remove_columns=drug_dataset["train"].column_names,
)
```

6.3 Qua lại với Pandas

`set_format("pandas")` đổi *định dạng xuất* sang DataFrame (dữ liệu nền vẫn là Arrow), tiện khi cần các phép phân tích mạnh của Pandas. Xong thì `reset_format()` để quay lại.

```
drug_dataset.set_format("pandas")
train_df = drug_dataset["train"][:]
frequencies = train_df["condition"].value_counts().to_frame().reset_index()
# ... và tạo lại Dataset từ DataFrame:
from datasets import Dataset
freq_dataset = Dataset.from_pandas(frequencies)
drug_dataset.reset_format()
```

6.4 Tạo tập validation và lưu dữ liệu

Nên để tập test "yên" và tạo một tập validation riêng khi phát triển, tránh quá khớp vào tập test. Hàm `train_test_split` (giống scikit-learn) lo việc chia dữ liệu:

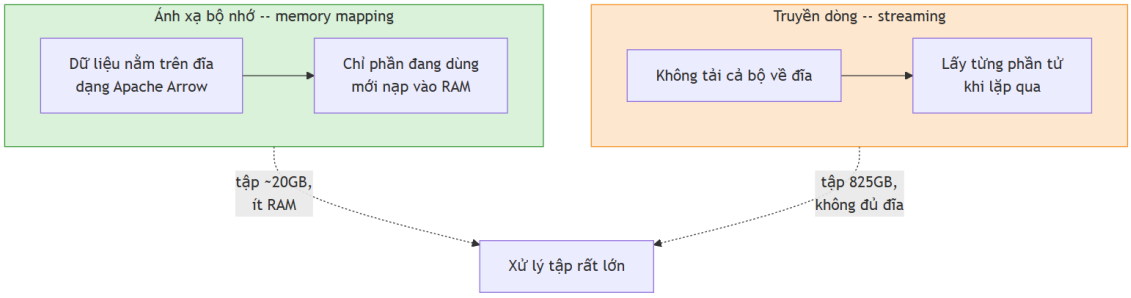
```
clean = drug_dataset["train"].train_test_split(train_size=0.8, seed=42)
clean["validation"] = clean.pop("test")      # đổi tên "test" -> "validation"
clean["test"] = drug_dataset["test"]          # giữ tập test gốc
```

Lưu dữ liệu ra ba định dạng: Arrow (`save_to_disk`), CSV (`to_csv`), JSON (`to_json`). Nạp lại Arrow bằng `load_from_disk`.

```
clean.save_to_disk("drug-reviews")
from datasets import load_from_disk
reloaded = load_from_disk("drug-reviews")
```

6.5 Dữ liệu lớn: ánh xạ bộ nhớ và truyền dòng

Hai cơ chế giúp xử lý dữ liệu vượt xa dung lượng RAM/đĩa (Hình 6.2).



Hình 6.2: Ánh xạ bộ nhớ và truyền dòng giúp làm việc với tập cực lớn.

Ánh xạ bộ nhớ (memory mapping): `datasets` coi mỗi bộ dữ liệu là tệp Arrow trên đĩa, chỉ nạp phần đang cần vào RAM. Nhờ vậy một tập gần 20 GB (ví dụ PubMed Abstracts, hơn 15 triệu dòng) vẫn mở và truy cập được dù RAM nhỏ.

Truyền dòng (streaming): bật `streaming=True` để lấy từng phần tử khi cần, không tải cả bộ về đĩa – cách duy nhất khả thi với tập như the Pile (825 GB). Đối tượng trả về là `IterableDataset`, phải lặp qua để lấy phần tử.

```
streamed = load_dataset("json", data_files=data_files, split="train", streaming=True)
next(iter(streamed))                  # lấy phần tử đầu

# vẫn map được khi truyền dòng (ví dụ token hoá):
tokenized = streamed.map(lambda x: tokenizer(x["text"]))

# trộn trong một bộ đệm, lấy/bỏ phần đầu:
shuffled = streamed.shuffle(buffer_size=10_000, seed=42)
val = shuffled.take(1000)             # 1000 phần tử đầu làm validation
train = shuffled.skip(1000)           # phần còn lại làm train
```

Có thể gộp nhiều bộ truyền dòng bằng `interleave_datasets` – xen kẽ các phần tử từ nhiều nguồn thành một `IterableDataset` duy nhất.

Tóm tắt chương

- `map`, `filter`, `sort`, `shuffle`, `select`, `rename_column` là bộ công cụ làm sạch và biến đổi dữ liệu.
- `map(batched=True)` nhanh hơn nhiều và còn thay đổi được số phần tử.
- `set_format("pandas")` cho phép dùng sức mạnh Pandas rồi quay lại Arrow.
- `train_test_split` tạo tập validation; lưu/nạp bằng Arrow, CSV, JSON.
- Dữ liệu lớn: ánh xạ bộ nhớ (ít RAM) và truyền dòng (không cần tải hết về đĩa).

Chương 7

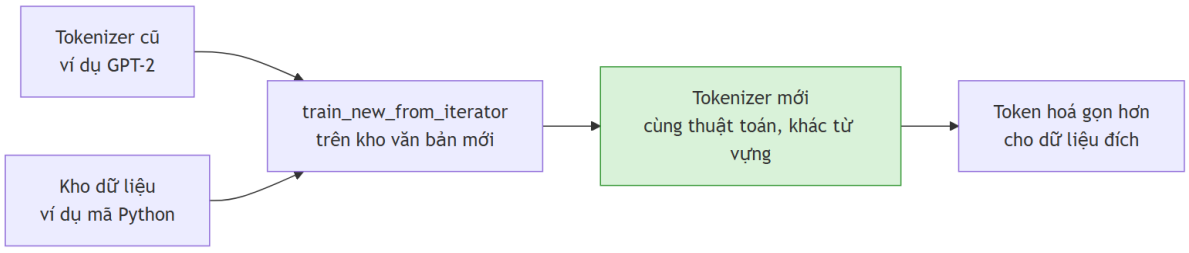
Thư viện Tokenizers

Chương 3 đã dùng tokenizer ở mức cơ bản. Chương này đi sâu hơn: huấn luyện một tokenizer mới cho dữ liệu của bạn, và khai thác những "siêu năng lực" của tokenizer nhanh.

7.1 Huấn luyện tokenizer mới từ một tokenizer cũ

Nếu không có mô hình cho ngôn ngữ bạn cần, hoặc kho văn bản của bạn khác xa dữ liệu gốc, bạn sẽ muốn huấn luyện lại với một tokenizer hợp dữ liệu. "Huấn luyện tokenizer" là quá trình *thống kê*: duyệt toàn bộ văn bản để tìm những từ phụ (subword) xuất hiện nhiều nhất. Khác với huấn luyện mô hình, đây là quá trình *tất định* – cùng thuật toán và cùng kho dữ liệu luôn cho cùng kết quả.

Cách gọn nhất là huấn luyện tokenizer mới mang *đặc tính* giống một tokenizer có sẵn (giữ nguyên thuật toán và token đặc biệt, chỉ đổi từ vựng) bằng `train_new_from_iterator` (Hình 7.1).



Hình 7.1: Huấn luyện tokenizer mới từ tokenizer cũ trên kho dữ liệu đích.

7.1.1 Chuẩn bị kho dữ liệu

Ví dụ: huấn luyện tokenizer cho mã Python, dùng bộ CodeSearchNet. Nên đưa dữ liệu dưới dạng **generator** để không nạp hết vào RAM – và bọc trong một hàm vì generator chỉ dùng được một lần.

```
from datasets import load_dataset

raw_datasets = load_dataset("code_search_net", "python")

def get_training_corpus():
    dataset = raw_datasets["train"]
    for start_idx in range(0, len(dataset), 1000):
        samples = dataset[start_idx : start_idx + 1000]
        yield samples["whole_func_string"]

training_corpus = get_training_corpus()
```

7.1.2 Huấn luyện và lưu

Nạp tokenizer cũ (GPT-2) rồi huấn luyện lại với từ vựng kích thước 52000:

```
from transformers import AutoTokenizer

old_tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer = old_tokenizer.train_new_from_iterator(training_corpus, 52000)

tokenizer.save_pretrained("code-search-net-tokenizer")
# chia sẻ lên Hub:
# tokenizer.push_to_hub("code-search-net-tokenizer")
```

Sau khi huấn luyện trên mã Python, tokenizer mới học được các token rất riêng cho mã: ví dụ một token biểu diễn cả mức thụt đầu dòng, một token cho ba dấu nháy mở docstring, và nó tách đúng tên hàm ở dấu gạch dưới. Nhờ vậy biểu diễn gọn hơn hẳn: cùng một ví dụ, bản mới cho **27** token so với **36** token của tokenizer tiếng Anh gốc.

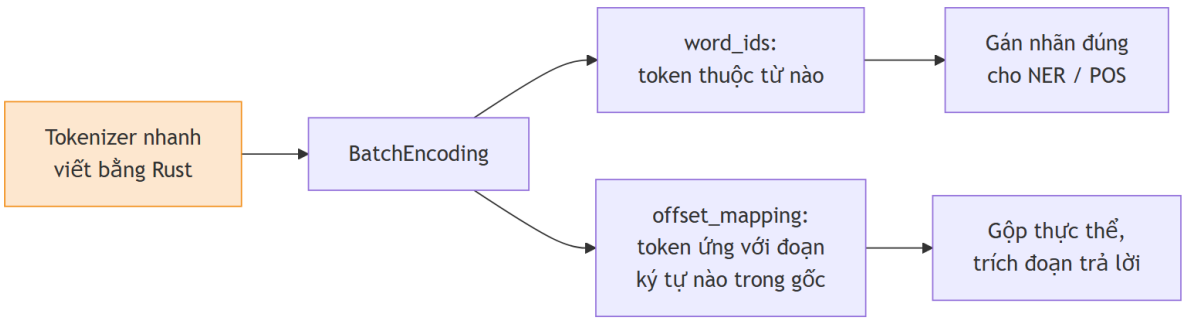
Lưu ý

`train_new_from_iterator` chỉ chạy với *tokenizer nhanh*. Phần huấn luyện chạy bằng Rust nên rất nhanh; nếu viết thuần Python sẽ chậm khủng khiếp.

7.2 Siêu năng lực của tokenizer nhanh

Thư viện có hai loại tokenizer: **chậm** (viết bằng Python) và **nhanh** (viết bằng Rust qua thư viện Tokenizers). `AutoTokenizer` mặc định chọn bản nhanh nếu có. Khác biệt tốc độ chỉ rõ khi token hoá *nhiều* văn bản song song.

Đầu ra của tokenizer không phải từ điển thường mà là đối tượng `BatchEncoding` với các phương thức bổ sung. Năng lực then chốt là **bản đồ offset** (offset mapping): tokenizer nhanh luôn ghi nhớ mỗi token sinh ra từ đoạn văn bản gốc nào (Hình 7.2).



Hình 7.2: Tokenizer nhanh và các khả năng mở ra nhờ bản đồ offset.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
example = "My name is Sylvain and I work at Hugging Face in Brooklyn."
encoding = tokenizer(example)

encoding.is_fast                # True
encoding.tokens()               # ['[CLS]','My',...,'S','##yl','##va','##in',...]
encoding.word_ids()             # token thứ i thuộc từ nào (None cho token đặc biệt)
```

```
start, end = encoding.word_to_chars(3)
example[start:end]              # 'Sylvain'
```

`word_ids` cho biết token thuộc từ nào – rất hữu ích để gán nhãn đúng cho từng token trong NER/POS, hoặc che cả một từ trong mô hình ngôn ngữ che từ.

7.2.1 Dựng lại pipeline phân loại token

Để thấy bản đồ offset hữu dụng thế nào, hãy tái tạo pipeline NER thủ công. Trước hết token hoá và chạy mô hình phân loại token:

```
import torch
from transformers import AutoTokenizer, AutoModelForTokenClassification

ckpt = "dbmdz/bert-large-cased-finetuned-conll03-english"
tokenizer = AutoTokenizer.from_pretrained(ckpt)
model = AutoModelForTokenClassification.from_pretrained(ckpt)

inputs = tokenizer(example, return_tensors="pt")
outputs = model(**inputs)
# logits có hình 1 x 19 x 9 -> 1 câu, 19 token, 9 nhãn
probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)[0].tolist()
predictions = outputs.logits.argmax(dim=-1)[0].tolist()
```

Nhãn theo định dạng IOB: `O` nghĩa là token không thuộc thực thể nào; `B-XXX` là token mở đầu thực thể loại XXX; `I-XXX` là token nằm bên trong thực thể XXX. Dùng `model.config.id2label` để đổi chỉ số thành tên nhãn.

Để lấy vị trí ký tự của mỗi thực thể trong câu gốc, bật `return_offsets_mapping=True`: mỗi token kèm một cặp (bắt đầu, kết thúc), còn token đặc biệt là `(0, 0)`.

```
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
offsets = inputs_with_offsets["offset_mapping"]
# token '##yl' có offset (12, 14) -> example[12:14] == 'yl'
```

7.2.2 Gộp thực thể

Khi muốn gộp các token cùng một thực thể (ví dụ `Hu`, `##gging`, `Face` thành "Hugging Face"), bản đồ offset giúp bỏ hết luật riêng rườm rà: chỉ cần lấy đoạn văn bản gốc từ token đầu đến token cuối của nhóm. Kết quả khớp đúng pipeline dùng `aggregation_strategy="simple"`, với điểm số là trung bình điểm các token trong nhóm.

Các thuật toán token hoá từ phụ

- **BPE** (byte-pair encoding, dùng trong GPT-2): bắt đầu từ ký tự, lặp lại việc gộp cặp ký hiệu hay đi cùng nhau nhất thành một token mới.
- **WordPiece** (dùng trong BERT): tương tự BPE nhưng chọn cặp để gộp theo tiêu chí khác; token nằm giữa từ có tiền tố ##.
- **Unigram** (dùng trong SentencePiece, nhiều mô hình đa ngôn ngữ): bắt đầu từ từ vựng lớn rồi dần loại bỏ token ít hữu ích nhất.

Tóm tắt chương

- Huấn luyện tokenizer là quá trình thống kê, tất định, khác với huấn luyện mô hình.
- `train_new_from_iterator` tạo tokenizer mới cùng thuật toán, khác từ vựng, hợp dữ liệu đích hơn (ví dụ mã nguồn).
- Tokenizer nhanh (Rust) nhanh khi xử lý song song và cung cấp bản đồ offset.
- Bản đồ offset và `word_ids` giúp gán nhãn, gộp thực thể, trích đoạn trả lời mà không cần luật riêng cho từng loại tokenizer.
- Ba thuật toán từ phụ chính: BPE, WordPiece, Unigram.

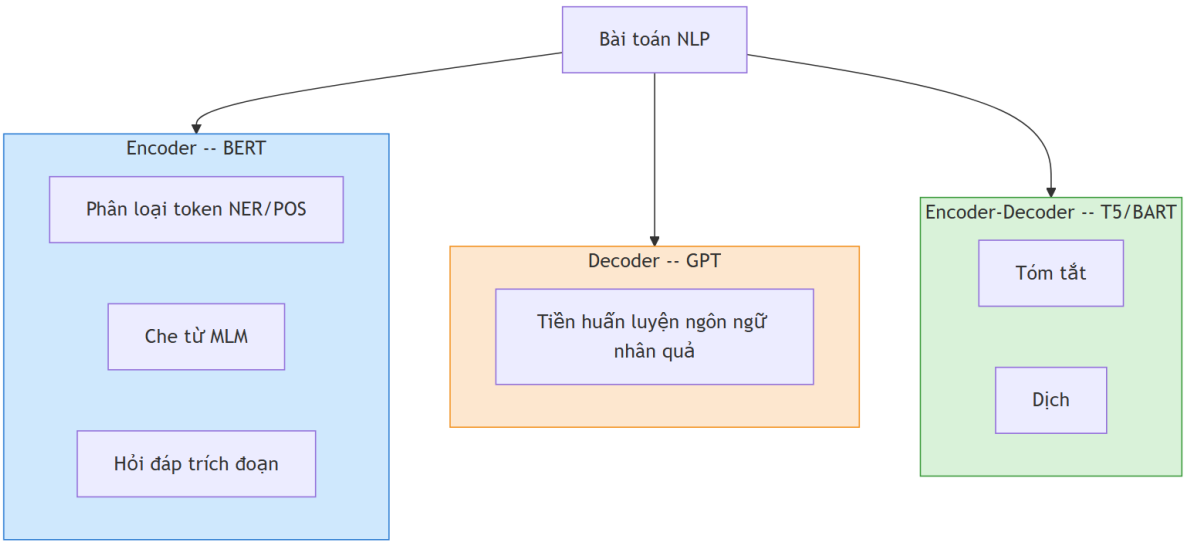
Chương 8

Các bài toán NLP kinh điển

Chương 4 đã tinh chỉnh mô hình cho phân loại văn bản. Chương này mở rộng sang các bài toán ngôn ngữ nền tảng – cũng chính là nền của cách LLM hiện đại hoạt động:

- Phân loại token (token classification).
- Mô hình ngôn ngữ che từ (masked language modeling, kiểu BERT).
- Tóm tắt (summarization).
- Dịch (translation).
- Tiền huấn luyện mô hình ngôn ngữ nhân quả (causal LM, kiểu GPT-2).
- Hỏi đáp (question answering).

Đây là nơi mọi thứ gặp nhau: API `Trainer` và Accelerate (Chương 4), thư viện `datasets` (Chương 6), thư viện `tokenizers` (Chương 7), và chia sẻ lên Hub (Chương 5). Mỗi tác vụ chọn kiến trúc phù hợp (Hình 8.1).

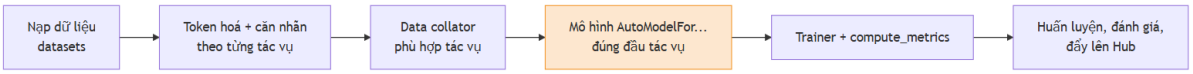


Hình 8.1: Mỗi nhóm bài toán hợp với một kiểu kiến trúc Transformer.

8.1 Công thức chung

Dù khác tác vụ, quy trình gần như giống nhau (Hình 8.2): nạp dữ liệu, token hoá và căn nhãn cho đúng tác vụ, chọn *data collator* phù hợp, nạp mô hình với đầu đúng tác vụ (`AutoModelFor...`), rồi huấn

luyện bằng `Trainer` kèm hàm độ đo. Nắm công thức này thì học tác vụ mới rất nhanh – chỉ thay vài chi tiết.



Hình 8.2: Công thức chung cho các bài toán NLP.

8.2 Phân loại token

Phân loại token gán nhãn cho *từng* token: nhận dạng thực thể tên riêng (NER), gán từ loại (POS), hay tách cụm. Dùng `AutoModelForTokenClassification`.

Thách thức chính: căn nhãn. Nhãn gốc gán theo *từ*, nhưng tokenizer tách từ thành nhiều token phụ. Dùng `word_ids` (Chương 7) để biết token thuộc từ nào, rồi lan nhãn của từ sang các token con; các token đặc biệt và token con phụ thường gán nhãn `-100` để bị bỏ qua khi tính độ lệch.

```
def align_labels_with_tokens(labels, word_ids):
    new_labels = []
    current_word = None
    for word_id in word_ids:
        if word_id != current_word:          # token đầu của một từ mới
            current_word = word_id
            new_labels.append(-100 if word_id is None else labels[word_id])
        elif word_id is None:                 # token đặc biệt
            new_labels.append(-100)
        else:                                 # token con tiếp theo của cùng từ
            label = labels[word_id]
            new_labels.append(label)
    return new_labels
```

Data collator dùng `DataCollatorForTokenClassification` (đệm cả nhãn). Độ đo thường dùng `seqeval` (precision, recall, F1 theo thực thể).

8.3 Mô hình ngôn ngữ che từ

Che một số token rồi bắt mô hình đoán lại – cách này giúp *thích nghi miền* (domain adaptation): lấy mô hình kiểu BERT rồi tiếp tục huấn luyện trên văn bản chuyên ngành của bạn. Dùng `AutoModelForMaskedLM` và `DataCollatorForLanguageModeling` với `mlm=True`; collator này tự che token ngẫu nhiên theo tỉ lệ `mlm_probability` mỗi lần tạo batch.

```
from transformers import DataCollatorForLanguageModeling

data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer, mlm=True, mlm_probability=0.15
)
```

Một biến thể là **che cả từ** (whole word masking): che mọi token con của cùng một từ, nhờ `word_ids`.

8.4 Tóm tắt và dịch

Cả hai là bài toán chuỗi-sang-chuỗi, dùng mô hình encoder-decoder (T5, BART, Marian). Bộ ba thành phần tương ứng:

- Lớp mô hình: `AutoModelForSeq2SeqLM`.
- Lớp huấn luyện: `Seq2SeqTrainer`.
- Collator: `DataCollatorForSeq2Seq`.

Điểm khác biệt: đầu vào và *đích* đều là văn bản, nên cần token hoá cả hai. Với T5, thường thêm tiền tố nhiệm vụ vào đầu vào (ví dụ "summarize: " hay "translate English to French: ").

```
def preprocess(examples):
    model_inputs = tokenizer(
        examples["input"], truncation=True, max_length=512
    )
    labels = tokenizer(
        text_target=examples["target"],
        truncation=True, max_length=128,
    )
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

Về độ đo: dùng **ROUGE** cho tóm tắt (độ trùng lặp với bản tóm tắt tham chiếu); dùng **BLEU** hoặc **SacreBLEU** cho dịch (độ trùng khớp cụm từ với bản dịch tham chiếu). Khi sinh để đánh giá, nhớ bật tuỳ chọn sinh văn bản (`predict_with_generate`) trong tham số huấn luyện seq2seq.

8.5 Tiền huấn luyện mô hình ngôn ngữ nhân quả

Huấn luyện mô hình kiểu GPT đoán từ kế tiếp. Dùng `AutoModelForCausalLM`. Dữ liệu thường được *gom và cắt khối* (chunking): nối văn bản rồi chia thành các khối cùng độ dài ngữ cảnh. Dùng `DataCollatorForLanguageModeling` với `mlm=False` – nhãn chính là đầu vào dịch sang phải một bước, collator lo giúp.

```
from transformers import DataCollatorForLanguageModeling

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
```

Đây là cách huấn luyện một mô hình sinh mã hay sinh văn bản chuyên miền từ (gần như) con số không, tận dụng tokenizer đã huấn luyện ở Chương 7.

8.6 Hỏi đáp

Hỏi đáp trích đoạn: cho câu hỏi và ngữ cảnh, mô hình dự đoán *vị trí bắt đầu* và *vị trí kết thúc* của câu trả lời trong ngữ cảnh. Dùng `AutoModelForQuestionAnswering`.

Thách thức: ngữ cảnh dài hơn giới hạn mô hình. Giải pháp là cắt ngữ cảnh thành nhiều đoạn chồng lấn bằng `return_overflowing_tokens=True` kèm `stride` (phần chồng lấn). Bản đồ offset (Chương 7) dùng để ánh xạ vị trí token dự đoán về vị trí ký tự trong văn bản gốc, và loại bỏ các câu trả lời rơi ngoài ngữ cảnh. Độ đo chuẩn trên SQuAD là **Exact Match** và **F1**.

Tóm tắt chương

- Mọi tác vụ theo cùng công thức: nạp dữ liệu, token hoá + căn nhãn, data collator, `AutoModelFor...`, `Trainer` + độ đo.
- Phân loại token cần căn nhãn theo `word_ids`; độ đo `seqeval`.
- Che từ (MLM) hợp để thích nghi miền; collator tự che token.
- Tóm tắt/dịch là seq2seq (T5, BART); độ đo ROUGE/BLEU, bật `predict_with_generate`.
- Tiền huấn luyện causal LM dùng gom-cắt khối và collator `mlm=False`.
- Hỏi đáp trích đoạn dự đoán vị trí bắt đầu/kết thúc; dùng offset và cửa sổ chồng lấn.

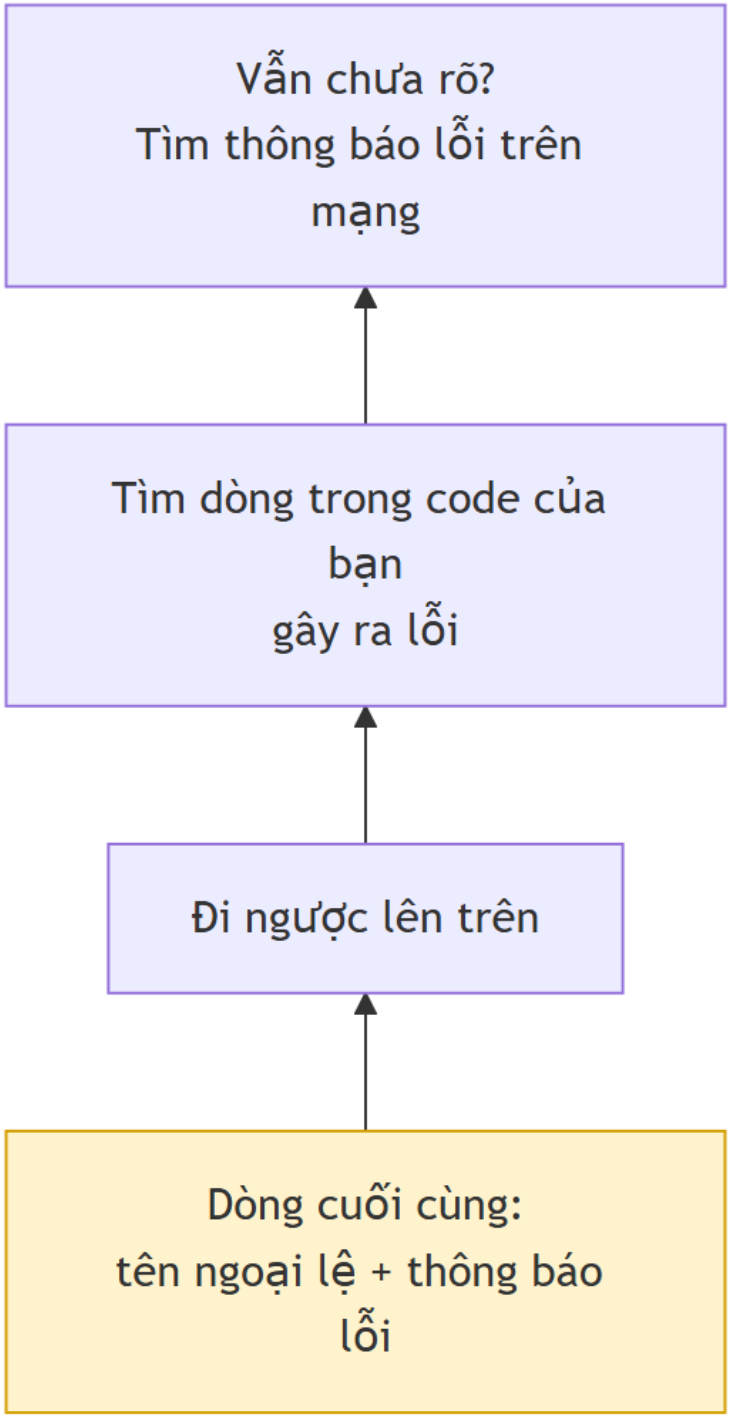
Chương 9

Khi cần trợ giúp

Lập trình với mô hình chắc chắn sẽ gặp lỗi. Chương này dạy cách đọc thông báo lỗi, gỡ lỗi có hệ thống cho pipeline suy luận và cho quá trình huấn luyện, và cách hỏi cộng đồng cho hiệu quả.

9.1 Đọc thông báo lỗi Python

Khi gặp lỗi, Python in ra một **traceback** (vết gọi hàm). Nguyên tắc quan trọng: *đọc từ dưới lên* (Hình 9.1). Dòng cuối cùng cho tên ngoại lệ và thông báo lỗi – thường chứa đủ manh mối. Nếu chưa đủ, đi ngược lên để tìm dòng trong code của bạn gây ra lỗi.



Hình 9.1: Cách đọc một traceback: từ dưới lên.

Mẹo

Nếu thông báo lỗi khó hiểu, cứ dán nguyên văn vào ô tìm kiếm của Google hay Stack Overflow. Khả năng cao bạn không phải người đầu tiên gặp lỗi đó.

9.2 Gỡ lỗi pipeline suy luận

Ví dụ: nạp một mô hình hỏi đáp từ Hub nhưng gặp `OSError: Can't load config`. Lần theo manh mối:

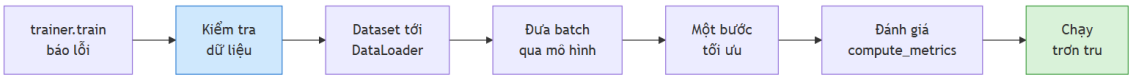
- **Kiểm tra tên mô hình**: dán ID vào ô tìm kiếm trên Hub – có thể chỉ là lỗi gõ (ví dụ "distillbert" thừa một chữ "l").
- **Kiểm tra nội dung kho**: dùng `list_repo_files` để xem kho có đủ tệp không. Lỗi trên là do thiếu tệp `config.json` – người tạo quên đẩy lên.

```
from huggingface_hub import list_repo_files
list_repo_files(repo_id=model_checkpoint)
# thiếu 'config.json' -> nạp cấu hình gốc rồi đẩy bổ sung:
from transformers import AutoConfig
config = AutoConfig.from_pretrained("distilbert-base-uncased")
config.push_to_hub(model_checkpoint, commit_message="Add config.json")
```

Một lỗi phổ biến khác khi tự chạy forward: `'list' object has no attribute 'size'`. Nguyên nhân: mô hình cần *tensor* nhưng ta đưa danh sách Python. Khắc phục: thêm `return_tensors="pt"` khi gọi tokenizer.

9.3 Gỡ lỗi quá trình huấn luyện

Khi `trainer.train()` báo lỗi, nguồn lỗi có thể ở nhiều nơi vì `Trainer` gộp rất nhiều khâu. Cách tốt nhất là đi qua từng khâu một cách thủ công (Hình 9.2).



Hình 9.2: Gỡ lỗi huấn luyện bằng cách đi qua từng khâu.

9.3.1 Kiểm tra dữ liệu

Trước hết nhìn vào *đúng* tập mà `Trainer` dùng (`trainer.train_dataset`). Một lỗi kinh điển: token hoá vào `tokenized_datasets` nhưng lại truyền `raw_datasets` cho `Trainer` – khiến mô hình nhận văn bản thô thay vì số. Luôn *giải mã* đầu vào để chắc chắn chúng có nghĩa:

```
tokenizer.decode(trainer.train_dataset[0]["input_ids"])
# kiểm tra attention_mask cùng độ dài với input_ids
# kiểm tra nhãn: so với features["label"].names để biết số nào là nhãn gì
```

9.3.2 Từ dataset tới dataloader

Nếu lỗi khi gom batch, thử tự tạo một batch:

```
for batch in trainer.get_train_dataloader():
    break
```

Lỗi kiểu "expected sequence of length X (got Y)" thường do collator sai: cần collator đệm động (`DataCollatorWithPadding`), nhưng `Trainer` chỉ tự tạo nó khi bạn truyền tokenizer vào. Giải pháp: luôn truyền collator một cách tường minh.

9.3.3 Đưa batch qua mô hình

Lỗi CUDA rất khó gỡ vì GPU chạy song song nên báo lỗi sai chỗ. Nguyên tắc: *đưa về CPU để gỡ* (trừ khi là lỗi hết bộ nhớ). Trên CPU, thông báo lỗi rõ hơn nhiều.

```
outputs = trainer.model.cpu()(**batch)
# ví dụ lỗi: "Target 2 is out of bounds" -> mô hình chỉ có 2 nhãn
# nhưng dữ liệu có 3 nhãn. Sửa: num_labels=3 khi tạo mô hình.
```

9.3.4 Một bước tối ưu và đánh giá

Hiếm khi lỗi ở bước `loss.backward()` và `optimizer.step()`; nếu có, lại đưa về CPU. Khâu đánh giá cũng hay lỗi: ví dụ `compute_metrics` nhận logits mà quên lấy `argmax` để ra nhãn dự đoán. Luôn chạy thử `trainer.evaluate()` trước khi `trainer.train()` để không phí tài nguyên.

Lỗi hết bộ nhớ GPU

Gặp `RuntimeError: CUDA out of memory`: giảm kích thước batch, không để hai mô hình trên GPU cùng lúc, hoặc dùng mô hình nhỏ hơn. Phải khởi động lại kernel để chạy tiếp.

9.4 Gỡ lỗi "im lặng": huấn luyện xong nhưng mô hình kém

Khó nhất là khi huấn luyện không báo lỗi nhưng mô hình học kém. Vài hướng:

- **Kiểm tra lại dữ liệu**: dữ liệu giải mã có hiểu được không? Nhãn có hợp lý không? Có nhãn nào áp đảo không? Loss/độ đo kỳ vọng của mô hình đoán ngẫu nhiên là bao nhiêu?
- **Thử quá khớp một batch**: huấn luyện lặp lại trên *đúng một batch* khoảng 20 bước. Nếu mô hình đạt gần như hoàn hảo trên batch đó thì bài toán và mã là khả thi; nếu không, có gì đó sai ở cách dựng bài toán hoặc dữ liệu.
- **Chưa có baseline thì chưa chỉnh siêu tham số**: mặc định của `Trainer` thường đã tốt. Chỉ dò siêu tham số sau khi đã vượt baseline, và mỗi lần chỉ đổi một tham số để biết cái nào tác động nhiều nhất.

9.5 Hỏi cộng đồng cho hiệu quả

Khi tự xử lý không xong, hãy hỏi trên diễn đàn Hugging Face. Một câu hỏi tốt cần: mô tả ngữ cảnh, đoạn code tối giản tái hiện lỗi, traceback đầy đủ (dạng văn bản, không phải ảnh), và những gì bạn đã thử. Càng dễ tái hiện, càng nhanh được giúp. Nếu nghi đây là lỗi của thư viện, hãy mở một issue mô tả rõ môi trường và cách tái hiện.

Tóm tắt chương

- Đọc traceback từ dưới lên; dán thông báo lỗi lên mạng để tìm giải pháp.
- Gỡ lỗi huấn luyện theo từng khâu: dữ liệu -> dataloader -> mô hình -> bước tối ưu -> đánh giá.
- Lỗi CUDA: đưa về CPU để có thông báo rõ ràng; lỗi hết bộ nhớ thì giảm batch.
- Mô hình kém dù không báo lỗi: kiểm tra dữ liệu, thử quá khớp một batch, có baseline rồi mới chỉnh siêu tham số.
- Hỏi cộng đồng với ví dụ tối giản, traceback đầy đủ và mô tả rõ ràng.

Chương 10

Dựng và chia sẻ demo

Có mô hình tốt rồi, bước tiếp theo là cho người khác thử. **Gradio** giúp dựng giao diện web quanh một hàm Python chỉ với vài dòng, rồi chia sẻ qua link tạm hoặc host lâu dài trên Hugging Face Spaces. Cài đặt đơn giản:

```
pip install gradio
```

10.1 Demo đầu tiên

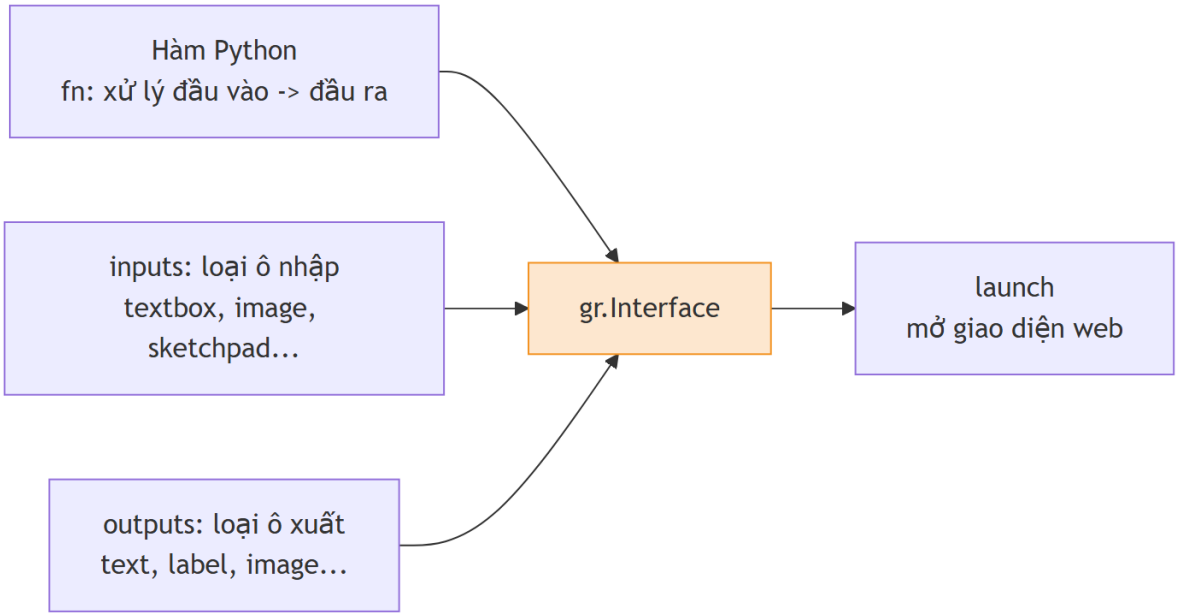
Lớp cốt lõi là `gr.Interface`, bọc một hàm Python thành giao diện. Ví dụ "Hello World":

```
import gradio as gr

def greet(name):
    return "Hello " + name

demo = gr.Interface(fn=greet, inputs="text", outputs="text")
demo.launch()
```

Ba đối số định hình giao diện (Hình 10.1): `fn` là hàm dự đoán, `inputs` là loại ô nhập, `outputs` là loại ô xuất. Gọi `launch()` để mở giao diện (trong notebook thì hiện ngay; chạy từ script thì mở ở `http://localhost:7860`).



Hình 10.1: Ba thành phần của một `gr.Interface`.

Muốn tuỳ biến ô nhập, truyền một đối tượng component thay vì chuỗi tắt:

```
textbox = gr.Textbox(label="Type your name here:", placeholder="John Doe", lines=2)
gr.Interface(fn=greet, inputs=textbox, outputs="text").launch()
```

10.1.1 Gắn mô hình thật vào demo

Hàm `fn` có thể là bất kỳ hàm Python nào – kể cả một pipeline của Transformers. Ví dụ demo sinh văn bản với GPT-2:

```
from transformers import pipeline
import gradio as gr

model = pipeline("text-generation")

def predict(prompt):
    completion = model(prompt)[0]["generated_text"]
    return completion

gr.Interface(fn=predict, inputs="text", outputs="text").launch()
```

10.2 Hiểu sâu lớp `Interface`

`Interface(fn, inputs, outputs, ...)` có ba đối số bắt buộc. Gradio cung cấp sẵn nhiều component – `"text"`, `"image"`, `"audio"`, `"label"`, `"mic"`… – và mỗi loại tuỳ biến được bằng cách khởi tạo lớp tương ứng.

10.2.1 Ví dụ với âm thanh

Demo đảo ngược âm thanh thu từ micro. Component `Audio` cho chọn nguồn (file hay micro) và kiểu dữ liệu; đặt `type="numpy"` để nhận dữ liệu dạng cặp (tần số lấy mẫu, mảng dữ liệu).

```
import numpy as np
import gradio as gr

def reverse_audio(audio):
    sr, data = audio
    reversed_audio = (sr, np.flipud(data))
    return reversed_audio

mic = gr.Audio(source="microphone", type="numpy", label="Speak here...")
gr.Interface(reverse_audio, mic, "audio").launch()
```

10.2.2 Nhiều đầu vào và đầu ra

Truyền *danh sách* component cho `inputs`/`outputs`: mỗi component ứng với một tham số (theo thứ tự) hoặc một giá trị trả về. Ví dụ sinh một nốt nhạc từ ba đầu vào:

```
import numpy as np
import gradio as gr

notes = ["C","C#","D","D#","E","F","F#","G","G#","A","A#","B"]

def generate_tone(note, octave, duration):
    sr = 48000
    a4_freq, tones_from_a4 = 440, 12 * (octave - 4) + (note - 9)
    frequency = a4_freq * 2 ** (tones_from_a4 / 12)
    duration = int(duration)
    audio = np.linspace(0, duration, duration * sr)
    audio = (20000 * np.sin(audio * (2 * np.pi * frequency))).astype(np.int16)
    return (sr, audio)

gr.Interface(
    generate_tone,
    [
        gr.Dropdown(notes, type="index"),
        gr.Slider(minimum=4, maximum=6, step=1),
        gr.Number(value=1, label="Duration in seconds"),
    ],
    "audio",
).launch()
```

10.2.3 Phương thức `launch`

Mặc định `launch()` chạy demo trên máy chủ web cục bộ. Vài tham số hữu ích: `inline` (nhúng trong notebook), `inbrowser` (tự mở tab trình duyệt), `share` (tạo link công khai – sẽ nói ở phần sau).

Ví dụ demo nhận dạng tiếng nói, nhận cả micro lẫn file tải lên:

```
from transformers import pipeline
import gradio as gr

model = pipeline("automatic-speech-recognition")

def transcribe_audio(audio):
    return model(audio)["text"]

gr.Interface(
    fn=transcribe_audio,
    inputs=gr.Audio(type="filepath"),
    outputs="text",
```

```
).launch()
```

10.3 Hoàn thiện và chia sẻ demo

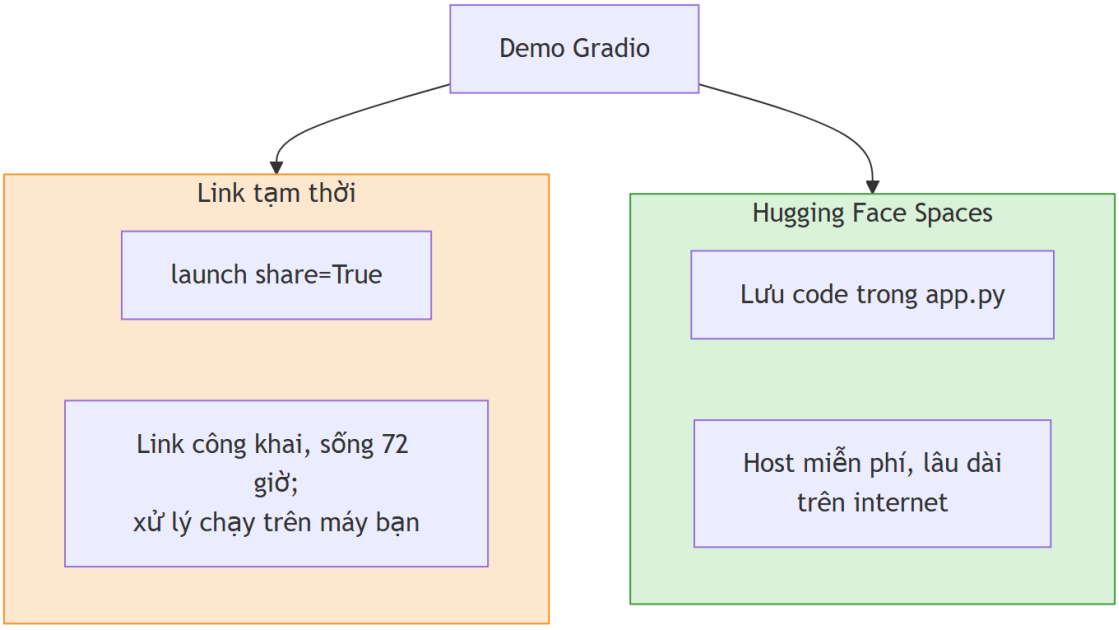
10.3.1 Làm đẹp giao diện

`Interface` nhận thêm vài tham số tuỳ chọn: `title` (tiêu đề), `description` (mô tả, hỗ trợ Markdown/HTML), `article` (bài viết dài bên dưới), `theme` (giao diện màu), `examples` (ví dụ đầu vào mẫu), `live` (chạy lại mỗi khi đầu vào đổi, không cần nút gửi).

```
gr.Interface(
    fn=predict,
    inputs="textbox",
    outputs="text",
    title="Ask Rick a Question",
    description="Ask Rick anything!",
    examples=[["What are you doing?"], ["Where should we time travel to?"]],
).launch()
```

10.3.2 Hai cách chia sẻ

Có hai cách đưa demo tới người khác (Hình 10.2).



Hình 10.2: Chia sẻ demo: link tạm thời hoặc host lâu dài trên Spaces.

Link tạm thời: đặt `share=True` trong `launch()`. Gradio tạo một link công khai (có dạng một địa chỉ đuôi `gradio.app`) sống tối đa 72 giờ; việc xử lý vẫn chạy trên máy bạn (máy phải bật). Trên Colab, link chia sẻ luôn được tạo tự động.

```
gr.Interface(classify_image, "image", "label").launch(share=True)
```

Cẩn trọng

Link chia sẻ là công khai – ai có link đều dùng mô hình của bạn được. Đừng để lộ thông tin nhạy cảm hay cho phép thay đổi quan trọng trên máy. Mặc định `share=False` chỉ tạo link cục bộ.

Host trên Spaces: Hugging Face Spaces host demo Gradio lâu dài, miễn phí. Tạo một kho (công khai hoặc riêng tư), đặt code giao diện trong tệp `app.py` là xong.

Tóm tắt chương

- `gr.Interface(fn, inputs, outputs)` dựng demo quanh bất kỳ hàm Python nào.
- Component có sẵn (text, image, audio, label…) và tuỳ biến bằng cách khởi tạo lớp tương ứng.
- Truyền danh sách component để có nhiều đầu vào/đầu ra; `live=True` để chạy lại tức thì.
- Chia sẻ nhanh bằng `launch(share=True)` (link sống 72 giờ), hoặc host lâu dài trên Spaces qua tệp `app.py`.
- Lưu ý bảo mật khi mở link công khai.

Chương 11

Tuyển chọn dữ liệu chất lượng

Chìa khoá để mô hình chạy tốt là *dữ liệu chất lượng cao*. Dữ liệu trên Hub có thể không hợp bài toán cụ thể của bạn, nên nhiều khi phải tự xây và tuyển chọn. Chương này dùng **Argilla** – công cụ chú thích (annotation) và tuyển chọn dữ liệu – để biến dữ liệu thô thành bộ dữ liệu tốt cho huấn luyện và đánh giá.

Với Argilla, bạn có thể: biến dữ liệu phi cấu trúc thành có cấu trúc; nâng chất lượng bộ dữ liệu; thu thập phản hồi của con người cho LLM; mời chuyên gia cùng chú thích hoặc huy động cộng đồng. Quy trình tổng quát ở Hình 11.1.



Hình 11.1: Quy trình tuyển chọn dữ liệu với Argilla.

11.1 Dựng và kết nối Argilla

Cách dễ nhất để chạy một bản Argilla là qua Hugging Face Spaces (có mẫu Space sẵn). Nên bật *lưu trữ bền vững* (persistent storage) để không mất dữ liệu khi Space tạm dừng. Sau đó cài SDK và kết nối từ Python:

```
pip install argilla
```

Cần ba thông tin để kết nối: **API URL** (địa chỉ Argilla đang chạy), **API key** (lấy trong "My Settings"), và **HF token** (chỉ cần nếu Space riêng tư).

```
import argilla as rg

HF_TOKEN = "..."  # chỉ cần cho Space riêng tư

client = rg.Argilla(
    api_url="...",
    api_key="...",
    headers={"Authorization": f"Bearer {HF_TOKEN}"},
)
client.me          # kiểm tra kết nối: trả về thông tin người dùng
```

11.2 Nạp và cấu hình dữ liệu

Ví dụ dùng bộ tin tức AG News cho hai tác vụ: phân loại chủ đề và nhận dạng thực thể. Trước hết xem đặc trưng dữ liệu để cấu hình đúng:

```
from datasets import load_dataset

data = load_dataset("SetFit/ag_news", split="train")
data.features
# {'text': ..., 'label': ..., 'label_text': ...}
```

Settings mô tả tác vụ chú thích, gồm *fields* (dữ liệu sẽ chú thích) và *questions* (các câu hỏi/tác vụ cần làm):

```
settings = rg.Settings(
    fields=[rg.TextField(name="text")],
    questions=[
        rg.LabelQuestion(
            name="label", title="Classify the text:",
            labels=data.unique("label_text"),
        ),
        rg.SpanQuestion(
            name="entities",
            title="Highlight all the entities in the text:",
            labels=["PERSON", "ORG", "LOC", "EVENT"],
            field="text",
        ),
    ],
)
```

Ở đây: `TextField` chứa văn bản cần chú thích; `LabelQuestion` cho phân loại (dùng chính các nhãn có sẵn trong dữ liệu để tương thích); `SpanQuestion` cho tác vụ gán nhãn khoảng (thực thể) trên trường văn bản.

11.2.1 Tạo dataset và nạp bản ghi

```
dataset = rg.Dataset(name="ag_news", settings=settings)
dataset.create()

# nạp dữ liệu làm bản ghi; ánh xạ cột khác tên giữa Hub và Argilla
dataset.records.log(data, mapping={"label_text": "label"})
```

Việc ánh xạ `label_text` sang câu hỏi `label` giúp dùng nhãn sẵn có làm *gợi ý trước* (pre-annotation), nhờ đó chú thích nhanh hơn.

11.3 Chú thích dữ liệu

11.3.1 Thống nhất bằng guidelines

Trước khi chú thích, nhất là khi làm nhóm, nên viết **hướng dẫn** (guidelines): mô tả tác vụ, cách dùng từng nhãn, cách xử lý tình huống mập mờ. Trong Argilla, sửa guidelines và mô tả câu hỏi ngay ở trang cài đặt dataset.

11.3.2 Phân phối công việc

Cài đặt phân phối quyết định mỗi bản ghi cần bao nhiêu phản hồi mới coi là hoàn tất. Mặc định là 1. Nếu muốn phân tích mức đồng thuận giữa người chú thích, đặt số lớn hơn (nhưng không vượt tổng số người chú thích). Làm một mình thì để 1.

11.3.3 Chú thích từng bản ghi

Mở dataset trên giao diện: câu hỏi phân loại đã điền sẵn nhãn gợi ý (nhờ ánh xạ ở trên) – chỉ cần xem lại và sửa. Với gán nhãn thực thể thì tự tô các khoảng. Mỗi bản ghi có ba hành động: **gửi** (submit) khi xong, **lưu nháp** (save as draft) để quay lại sau, hoặc **bỏ** (discard) nếu không dùng.

Tóm tắt chương

- Dữ liệu chất lượng cao là yếu tố quyết định; Argilla giúp chú thích và tuyển chọn.
- Dựng Argilla qua Spaces (nhớ bật lưu trữ bền vững), kết nối bằng SDK với API URL, API key, và HF token.
- `Settings` gồm *fields* và *questions*; ánh xạ nhãn sẵn có để tạo gợi ý trước, chú thích nhanh hơn.
- Viết guidelines rõ ràng và cấu hình phân phối công việc khi làm nhóm.
- Sau khi chú thích, xuất bộ dữ liệu chất lượng cao lên Hub để huấn luyện/đánh giá.

Chương 12

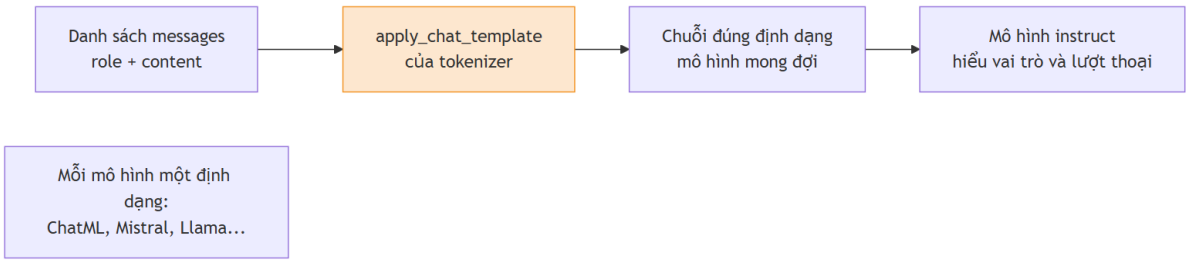
Tinh chỉnh mô hình ngôn ngữ lớn

Chương này đi vào tinh chỉnh LLM theo hướng hội thoại: định dạng chat chuẩn, tinh chỉnh có giám sát (SFT), và kỹ thuật tiết kiệm tham số (LoRA). Công cụ chính là thư viện **TRL** (xây trên `transformers`) và **PEFT**.

12.1 Mẫu hội thoại (chat templates)

Mẫu hội thoại định hình cách mô hình và người dùng trao đổi: giữ cấu trúc nhất quán, xác định vai trò (hệ thống, người dùng, trợ lý), quản lý ngữ cảnh nhiều lượt, và hỗ trợ gọi công cụ.

Mô hình nền (base) chỉ học đoán từ kế tiếp trên văn bản thô; **mô hình instruct** được tinh chỉnh để bám chỉ dẫn và trò chuyện. Để một mô hình xử lý hội thoại đúng, prompt phải được định dạng theo đúng mẫu nó mong đợi (Hình 12.1). Mỗi họ mô hình có định dạng riêng (ChatML, Mistral, Llama…).



Hình 12.1: Mẫu hội thoại biến danh sách tin nhắn thành chuỗi mô hình hiểu được.

Điểm hay: không cần tự viết định dạng. Tokenizer của mỗi mô hình đã nhúng sẵn mẫu; chỉ cần gọi `apply_chat_template` trên danh sách tin nhắn.

```
from transformers import AutoTokenizer

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"},
]

tokenizer = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM2-135M-Instruct")
chat = tokenizer.apply_chat_template(messages, tokenize=False)
```

Lưu ý

Dùng mô hình instruct thì luôn kiểm tra đúng mẫu hội thoại – dùng sai mẫu sẽ khiến mô hình chạy kém hoặc hành xử lạ. Cách chắc chắn nhất là xem cấu hình tokenizer của mô hình trên Hub.

12.2 Tinh chỉnh có giám sát (SFT)

SFT điều chỉnh mô hình nền để biết bám chỉ dẫn, trò chuyện và tuân định dạng đầu ra, bằng cách huấn luyện trên các cặp *prompt – phản hồi* do người viết.

12.2.1 Khi nào nên dùng SFT

SFT tốn tài nguyên và công sức, nên chỉ dùng khi *viết prompt khéo trên mô hình instruct có sẵn vẫn chưa đủ*. Cân nhắc SFT khi cần: hiệu năng vượt mức prompt đạt được; dùng mô hình nhỏ tinh chỉnh thay cho mô hình lớn tốn kém; hoặc cần định dạng đầu ra đặc thù/kiến thức chuyên ngành. Hai lợi ích chính của SFT là **kiểm soát định dạng** đầu ra và **thích nghi miền**.

12.2.2 Huấn luyện với `SFTTrainer`

Lớp `SFTTrainer` của TRL dùng tương tự `Trainer` ở Chương 4, cấu hình qua `SFTConfig`.

```
from datasets import load_dataset
from trl import SFTConfig, SFTTrainer

dataset = load_dataset("HuggingFaceTB/smoltalk", "all")

training_args = SFTConfig(
    output_dir="./sft_output",
    max_steps=1000,
    per_device_train_batch_size=4,
    learning_rate=5e-5,
    logging_steps=10,
    save_steps=100,
    eval_strategy="steps",
    eval_steps=50,
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    processing_class=tokenizer,
)
trainer.train()
```

Khi dữ liệu có trường `messages`, `SFTTrainer` tự áp mẫu hội thoại của mô hình – không cần cấu hình thêm. Bật `packing=True` trong `SFTConfig` để gói nhiều ví dụ ngắn vào một chuỗi, tận dụng GPU tốt hơn.

Các nhóm tham số huấn luyện quan trọng: thời lượng (`num_train_epochs` hay `max_steps`), kích thước batch (`per_device_train_batch_size`, `gradient_accumulation_steps`), tốc độ học (`learning_rate`, `warmup_ratio`), và theo dõi (`logging_steps`, `eval_steps`, `save_steps`). Nên bắt đầu dè dặt: 1–3 epoch, batch nhỏ, theo dõi sát độ đo trên tập validation.

12.2.3 Đọc đường cong loss

Loss huấn luyện thường qua ba pha: giảm mạnh lúc đầu, chậm dần, rồi ổn định (hội tụ). Dấu hiệu lành mạnh là khoảng cách nhỏ giữa loss huấn luyện và loss validation. Vài cảnh báo:

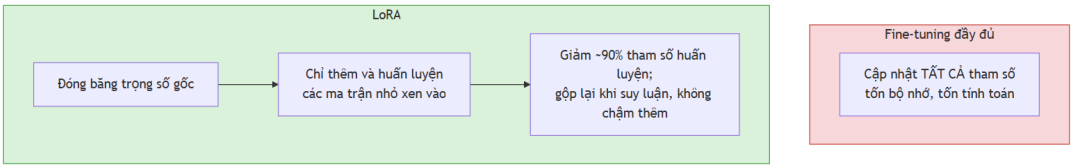
- Loss validation tăng trong khi loss huấn luyện giảm: dấu hiệu quá khớp (giảm số bước, tăng hoặc đa dạng hoá dữ liệu).

- Loss gần như không cải thiện: học quá chậm hoặc bài toán khó (tăng tốc độ học, kiểm tra chất lượng dữ liệu, hoặc đổi mô hình).
- Loss cực thấp bất thường: có thể là học thuộc lòng (kém trên ví dụ mới, đầu ra thiếu đa dạng).

Luôn theo dõi cả loss *và* đầu ra thực tế – đôi khi loss đẹp nhưng mô hình sinh ra hành vi không mong muốn.

12.3 LoRA: tinh chỉnh tiết kiệm tham số

Tinh chỉnh đầy đủ một LLM rất tốn tài nguyên. **LoRA** (Low-Rank Adaptation) đóng băng trọng số gốc và chỉ thêm *các ma trận nhỏ* xen vào (chủ yếu ở trọng số attention), nên giảm khoảng **90%** số tham số cần huấn luyện mà vẫn giữ hiệu năng (Hình 12.2). Khi suy luận, các trọng số thêm này gộp được vào mô hình gốc nên không gây chậm thêm.



Hình 12.2: Ý tưởng LoRA so với tinh chỉnh đầy đủ.

Lợi ích: tiết kiệm bộ nhớ (chỉ lưu tham số adapter, trọng số gốc đóng băng và nạp ở độ chính xác thấp hơn), tinh chỉnh được mô hình lớn trên GPU phổ thông, và dễ chuyển đổi giữa nhiều adapter cho các tác vụ khác nhau.

12.3.1 Cấu hình LoRA với PEFT

```
from peft import LoraConfig

peft_config = LoraConfig(
    r=6,                        # hạng của ma trận LoRA, thường 4-32
    lora_alpha=8,               # hệ số tỉ lệ, thường gấp 2 lần hạng
    lora_dropout=0.05,          # dropout, thường 0.05-0.1
    bias="none",
    target_modules="all-linear",   # áp LoRA vào những lớp nào
    task_type="CAUSAL_LM",
)
```

Vài tham số then chốt: `r` (hạng – nhỏ thì nén nhiều, biểu diễn kém hơn), `lora_alpha` (hệ số tỉ lệ – lớn thì thích nghi mạnh hơn), `lora_dropout` (chống quá khớp), và `target_modules` (lớp nào được gắn LoRA). Nên bắt đầu với hạng nhỏ (4–8) và theo dõi loss.

12.3.2 Dùng LoRA trong `SFTTrainer` và gộp adapter

Chỉ cần truyền `peft_config` vào `SFTTrainer`:

```
trainer = SFTTrainer(
    model=model,
    args=args,
    train_dataset=dataset["train"],
    peft_config=peft_config,
    processing_class=tokenizer,
)
```

Sau khi huấn luyện, có thể *gộp* trọng số adapter vào mô hình gốc để triển khai gọn (không cần nạp adapter riêng). Giữ nhất quán độ chính xác (ví dụ float16) và nhớ lưu cả tokenizer.

```
import torch
from transformers import AutoModelForCausalLM
from peft import PeftModel

base_model = AutoModelForCausalLM.from_pretrained(
    "base_model_name", torch_dtype=torch.float16, device_map="auto"
)
peft_model = PeftModel.from_pretrained(
    base_model, "path/to/adapter", torch_dtype=torch.float16
)
merged_model = peft_model.merge_and_unload()
```

Tóm tắt chương

- Mẫu hội thoại định dạng tin nhắn theo đúng kỳ vọng mô hình; dùng `apply_chat_template`, đừng tự viết tay.
- SFT dạy mô hình bám chỉ dẫn và định dạng; chỉ dùng khi prompt không đủ.
- `SFTTrainer` (TRL) + `SFTConfig` làm việc như `Trainer`; theo dõi đường cong loss để phát hiện quá khớp/kém học.
- LoRA đóng băng trọng số gốc, chỉ huấn luyện ma trận nhỏ – giảm khoảng 90% tham số, chạy được trên GPU phổ thông.
- Truyền `peft_config` vào `SFTTrainer`; gộp adapter để triển khai gọn.

Chương 13

Mô hình suy luận

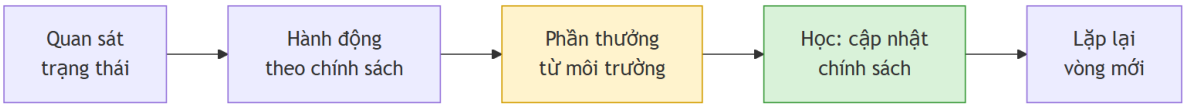
Chương cuối giới thiệu học tăng cường (Reinforcement Learning – RL) cho mô hình ngôn ngữ, và cách nó giúp xây mô hình biết *suy luận*. Trọng tâm là thuật toán GRPO (Group Relative Policy Optimization) và cách hiện thực bằng thư viện TRL.

13.1 Học tăng cường là gì

Hình dung bạn dạy một chú chó ngồi: nói "Ngồi!", nếu chó ngồi thì thưởng; lặp lại, chú chó dần học được. Đó là ý tưởng cốt lõi của RL. Thay chú chó bằng một LLM (**tác nhân** – agent), thay bạn bằng **môi trường** đưa phản hồi. Các khái niệm chính:

- **Tác nhân** (agent): người học – ở đây là LLM.
- **Môi trường** (environment): thế giới tác nhân tương tác – có thể là người dùng hay một tình huống mô phỏng.
- **Hành động** (action): lựa chọn của tác nhân – với LLM là sinh từ, chọn câu trả lời.
- **Phần thưởng** (reward): phản hồi (số) từ môi trường – dương là "làm tốt", âm là "chưa đúng".
- **Chính sách** (policy): chiến lược chọn hành động của tác nhân – chính là thứ ta muốn học và cải thiện.

RL diễn ra theo vòng lặp thử-và-sai (Hình 13.1): quan sát môi trường, hành động theo chính sách, nhận phần thưởng, cập nhật chính sách, rồi lặp lại – giống học đi xe đạp: ngã (thưởng âm), giữ thăng bằng tốt (thưởng dương), rồi điều chỉnh dần.



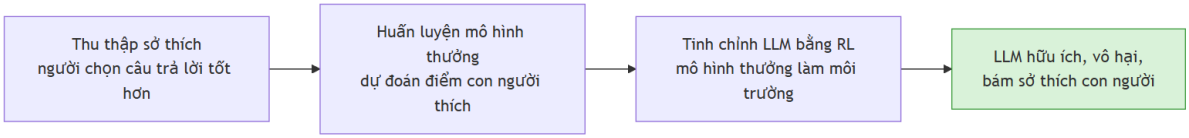
Hình 13.1: Vòng lặp học tăng cường.

13.2 Vì sao RL quan trọng với LLM

Tiền huấn luyện (đoán từ kế tiếp) giúp LLM viết trôi chảy, nhưng trôi chảy chưa đủ. Ta muốn LLM **hữu ích**, **vô hại**, và **bám sở thích con người**. RL cho phép tinh chỉnh để đạt những phẩm chất đó.

Kỹ thuật phổ biến là **RLHF** (Reinforcement Learning from Human Feedback) – dùng phản hồi của con người làm tín hiệu thưởng (Hình 13.2):

1. **Thu thập sở thích**: cho người so sánh các câu trả lời của LLM và chọn câu tốt hơn.
2. **Huấn luyện mô hình thưởng**: dùng dữ liệu sở thích để huấn luyện một mô hình học dự đoán điểm mà con người sẽ thích.
3. **Tinh chỉnh LLM bằng RL**: dùng mô hình thưởng làm môi trường chấm điểm các câu LLM sinh ra, nhờ đó LLM học sinh ra câu được chấm cao.



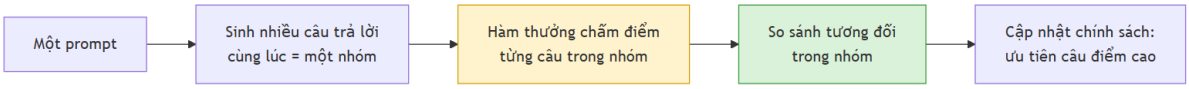
Hình 13.2: Ba bước của RLHF.

RLHF đã dùng để huấn luyện nhiều LLM nổi tiếng (GPT-4, Gemini, DeepSeek R1). Lợi ích: kiểm soát tốt hơn kiểu văn bản sinh ra, bám sát giá trị con người, và giảm hành vi không mong muốn (độc hại, sai lệch, thiên lệch).

13.3 GRPO và vì sao chọn nó

Có nhiều kỹ thuật RLHF: PPO (Proximal Policy Optimization) là một trong những kỹ thuật hiệu quả đầu tiên, dùng mô hình thưởng riêng; DPO (Direct Preference Optimization) đơn giản hơn, học thẳng từ dữ liệu sở thích mà không cần mô hình thưởng riêng.

Khoá học tập trung vào **GRPO** (Group Relative Policy Optimization). Khác hai kỹ thuật trên, GRPO *gom các mẫu tương tự thành nhóm và so sánh trong nhóm* (Hình 13.3): cách so sánh tương đối theo nhóm cho gradient ổn định hơn và hội tụ tốt hơn. GRPO cũng linh hoạt về nguồn tín hiệu thưởng – có thể là mô hình thưởng, hoặc *bất kỳ hàm nào* chấm được chất lượng câu trả lời (ví dụ hàm độ dài, bộ giải toán kiểm tra đáp án, hàm kiểm tra tính đúng đắn).



Hình 13.3: GRPO: sinh một nhóm câu trả lời rồi so sánh tương đối trong nhóm.

13.4 Hiện thực GRPO với TRL

Để chạy GRPO cần: một tập *prompt*, một *hàm thưởng* (nhận danh sách câu trả lời và trả về danh sách điểm), cấu hình `GRPOConfig`, và lớp `GRPOTrainer`.

```
from trl import GRPOTrainer, GRPOConfig
from datasets import load_dataset

dataset = load_dataset("your_dataset", split="train")

def reward_func(completions, **kwargs):
    """Ví dụ: thưởng câu trả lời dài hơn."""
    return [float(len(completion)) for completion in completions]

training_args = GRPOConfig(
    output_dir="output",
    num_train_epochs=3,
    per_device_train_batch_size=4,
```

```
    gradient_accumulation_steps=2,
    logging_steps=10,
)

trainer = GRPOTrainer(
    model="Qwen/Qwen2-0.5B-Instruct",
    args=training_args,
    train_dataset=dataset,
    reward_funcs=reward_func,
)
trainer.train()
```

13.4.1 Kích thước nhóm

Tham số `num_generation` định số câu trả lời sinh ra cho mỗi prompt – chính là *kích thước nhóm*, điểm khác biệt cốt lõi của GRPO. Quá nhỏ (2–3) thì thiếu đa dạng để so sánh; khuyến nghị 4–16 (cân bằng đa dạng và chi phí); lớn hơn có thể học tốt hơn nhưng tốn tính toán. Bài toán đơn giản dùng nhóm nhỏ (4–8), bài suy luận phức tạp dùng nhóm lớn hơn (8–16).

13.4.2 Thiết kế hàm thưởng

Hàm thưởng quyết định mô hình học gì. Vài kiểu lấy cảm hứng từ cách huấn luyện DeepSeek R1:

Thưởng theo độ dài: khuyến khích độ dài gần một mức lý tưởng.

```
def reward_len(completions, **kwargs):
    ideal_length = 20
    return [-abs(ideal_length - len(c)) for c in completions]
```

Thưởng theo luật (cho bài có đáp án kiểm chứng được, như toán hay lập trình): trích đáp án rồi cho điểm 1 nếu đúng, 0 nếu sai.

Thưởng theo định dạng: thưởng câu trả lời theo đúng khuôn "suy nghĩ rồi trả lời" – ví dụ có khối `<think>...</think>` rồi `<answer>...</answer>`. Đây là tín hiệu quan trọng để mô hình học *trình bày quá trình suy luận*.

```
import re

def format_reward(completions, **kwargs):
    pattern = r"<think>(.*?)</think>\s*<answer>(.*?)</answer>"
    rewards = []
    for c in completions:
        m = re.search(pattern, c, re.DOTALL)
        if m and len(m.group(1).strip()) > 20 and len(m.group(2).strip()) > 0:
            rewards.append(1.0)
        elif m:
            rewards.append(0.5)      # đúng định dạng nhưng nội dung sơ sài
        else:
            rewards.append(0.0)
    return rewards
```

Mẹo theo dõi

Khi huấn luyện GRPO, theo dõi: `reward` (điểm thưởng trung bình), `reward_std` (độ lệch trong nhóm), và `kl` (độ lệch so với mô hình tham chiếu). Bật `use_vllm=True` để sinh nhanh hơn nếu mô hình được hỗ trợ.

Tóm tắt chương

- RL dạy tác nhân qua thử-và-sai: quan sát, hành động, nhận thưởng, cập nhật chính sách.
- RLHF dùng phản hồi con người để làm LLM hữu ích, vô hại và bám sở thích.
- GRPO so sánh *tương đối trong nhóm* câu trả lời, cho gradient ổn định và linh hoạt về nguồn thưởng.
- TRL cung cấp `GRPOTrainer` + `GRPOConfig`; chỉ cần prompt, hàm thưởng và cấu hình.
- Hàm thưởng (độ dài, theo luật, theo định dạng) định hình điều mô hình học – kể cả thói quen trình bày quá trình suy luận.